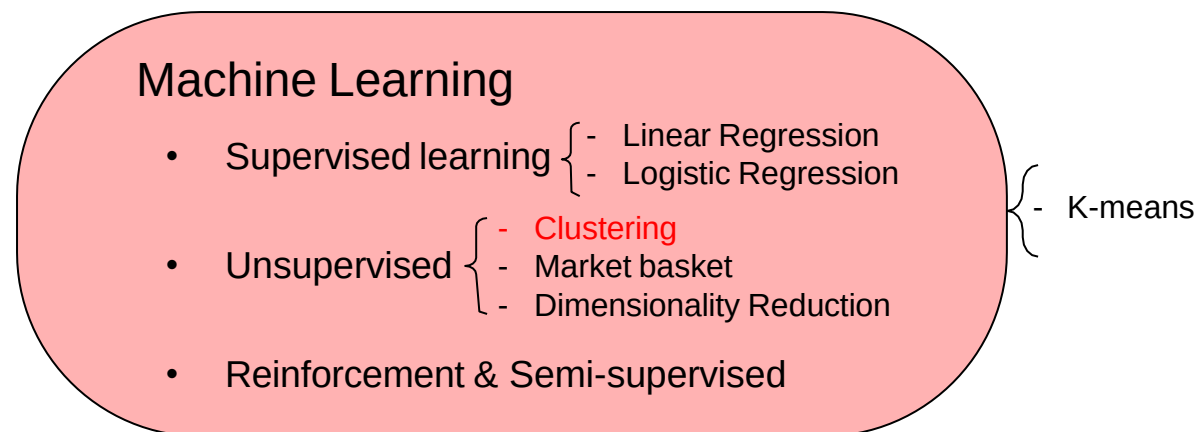


ML
Unsupervised Learning
(Clustering)

Unsupervised Learning

- **Definition** (Wikipedia)

Unsupervised learning is a type of machine learning that looks for previously undetected patterns in a data set with no pre-existing labels and with a minimum of human supervision.

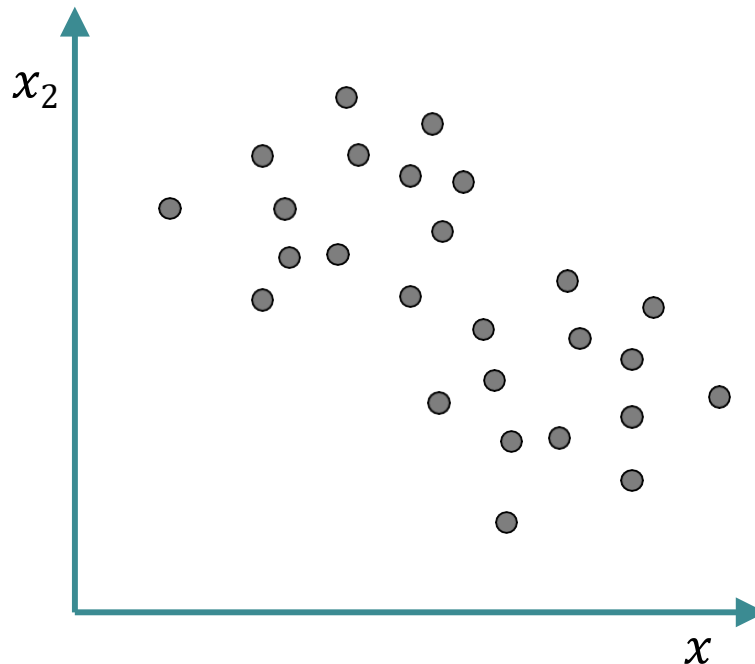


Clustering – K-Means

Clustering – K-Means

- Algorithm

- 1º Set the k-number of Cluster
- 2º Choice k points at random, the centroids
- 3º Assign each point to the nearest centroid to create K-clusters.
- 4º Calculate and assign the new centroid of each cluster.
- 5º reassign each point to its nearest centroid. If there have been new assignments go to step 4, otherwise finish.

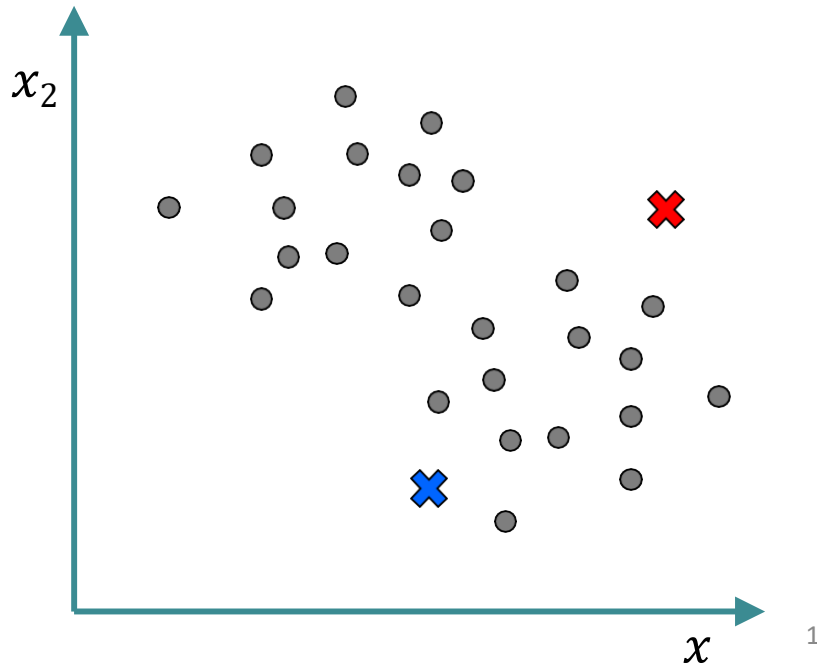


Step 1: K=2

Clustering – K-Means

- Algorithm

- 1º Set the k-number of Cluster
- 2º Choice k points at random, the centroids
- 3º Assign each point to the nearest centroid to create K-clusters.
- 4º Calculate and assign the new centroid of each cluster.
- 5º reassign each point to its nearest centroid. If there have been new assignments go to step 4, otherwise finish.



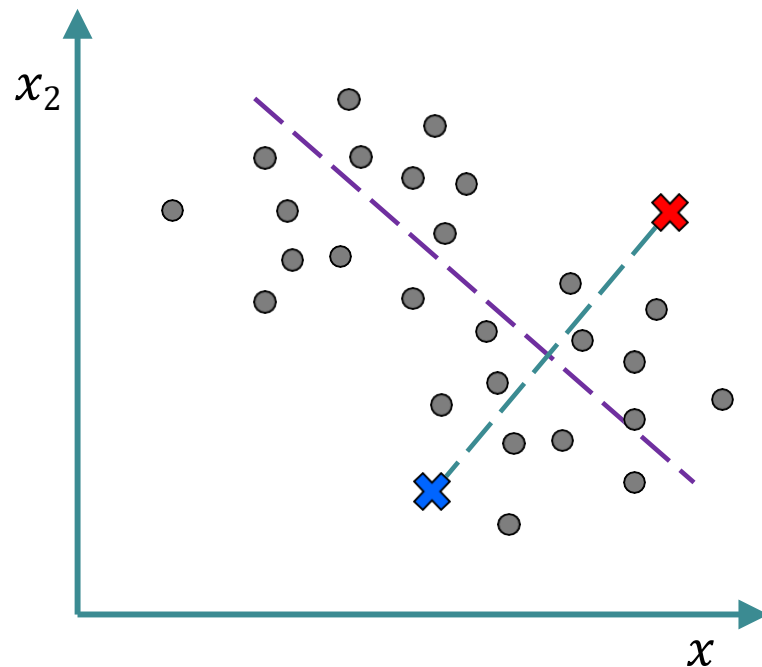
Step 2: Choice 2 points

Clustering – K-Means

- Algorithm

- 1º Set the k-number of Cluster
- 2º Choice k points at random, the centroids
- 3º Assign each point to the nearest centroid to create K-clusters.

- 
- 4º Calculate and assign the new centroid of each cluster.
 - 5º reassign each point to its nearest centroid. If there have been new assignments go to step 4, otherwise finish.



Step 3: Assign each point to the nearest centroid.

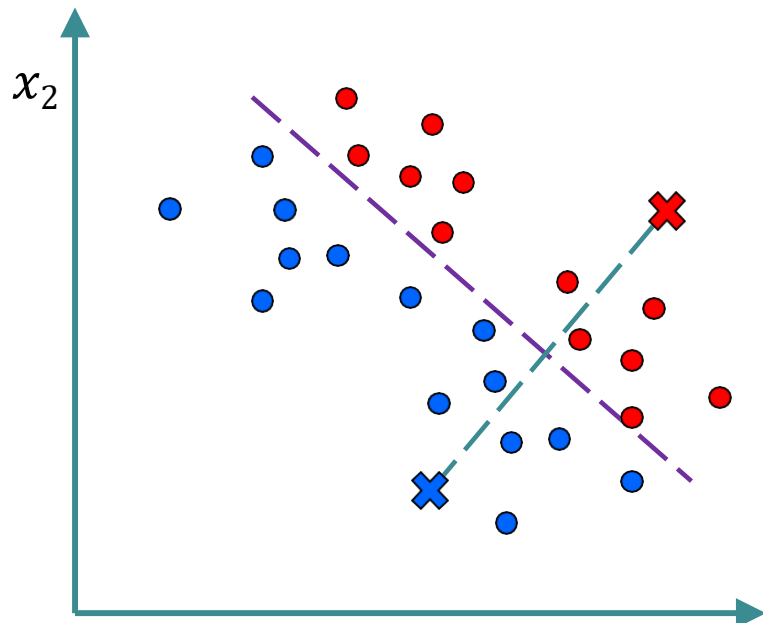
Clustering – K-Means

- Algorithm

- 1º Set the k-number of Cluster
- 2º Choice k points at random, the centroids
- 3º Assign each point to the nearest centroid to create K-clusters.



- 4º Calculate and assign the new centroid of each cluster.
- 5º reassign each point to its nearest centroid. If there have been new assignments go to step 4, otherwise finish.

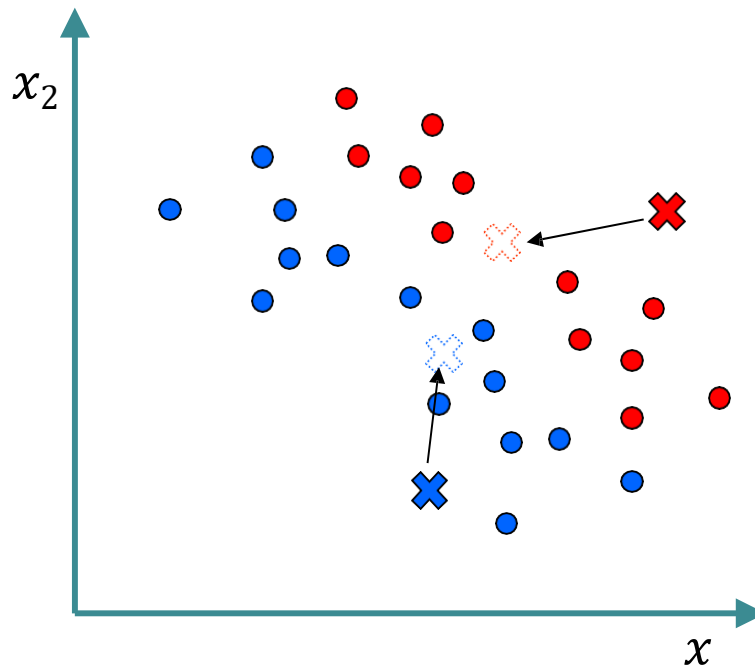


Step 3: Assign each point
to the nearest centroid.
To create 2-Clusters

Clustering – K-Means

- Algorithm

- 1º Set the k-number of Cluster
- 2º Choice k points at random, the centroids
- 3º Assign each point to the nearest centroid to create K-clusters.
- 4º Calculate and assign the new centroid of each cluster.
- 5º reassign each point to its nearest centroid. If there have been new assignments go to step 4, otherwise finish.



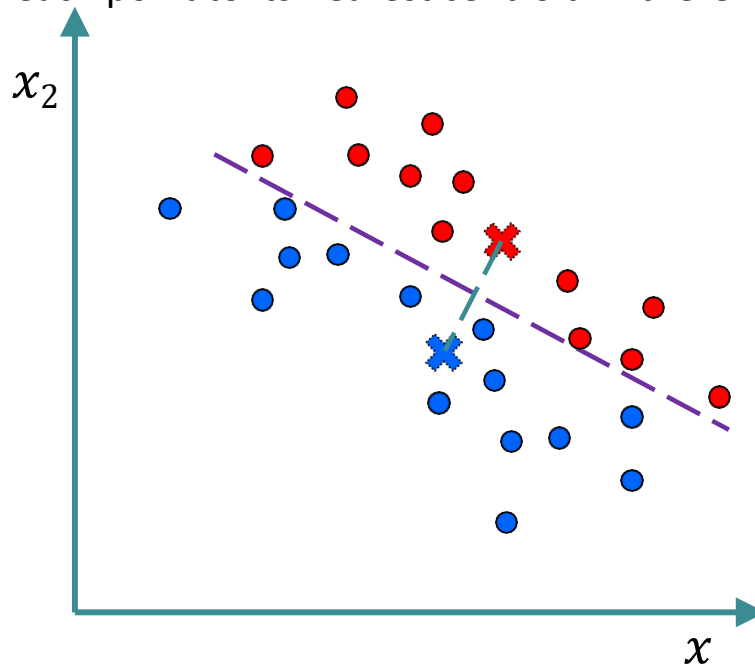
Step 4: Calculate and assign the new centroid

Clustering – K-Means

- Algorithm

- 1^o Set the k-number of Cluster
- 2^o Choice k points at random, the centroids
- 3^o Assign each point to the nearest centroid to create K-clusters.

- 4^o Calculate and assign the new centroid of each cluster.
- 5^o reassign each point to its nearest centroid. If there have been new assignments go to step 4, otherwise finish.



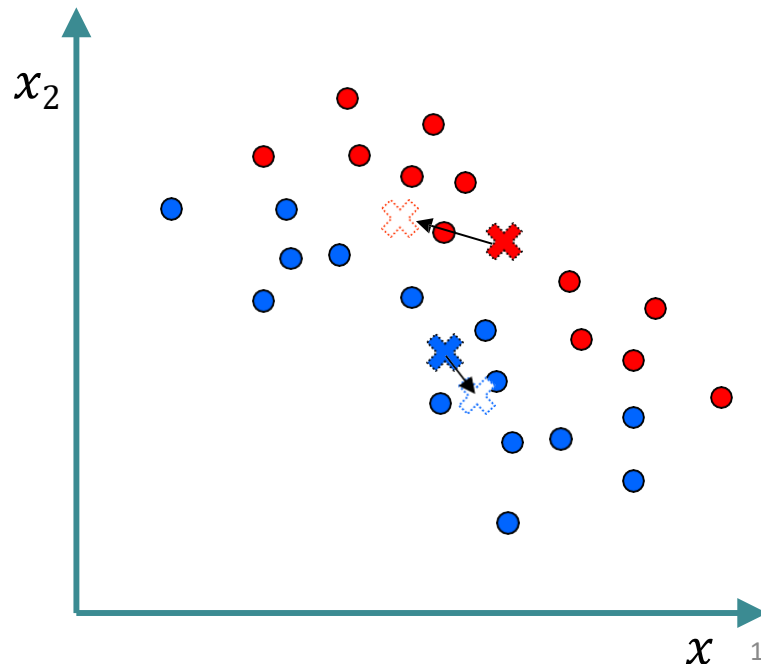
Step 5: reassign each point to its nearest centroid

Clustering – K-Means

- Algorithm

- 1º Set the k-number of Cluster
- 2º Choice k points at random, the centroids
- 3º Assign each point to the nearest centroid to create K-clusters.

- 
- 4º Calculate and assign the new centroid of each cluster.
 - 5º reassign each point to its nearest centroid. If there have been new assignments go to step 4, otherwise finish.



Step 4: Calculate and assign the new centroid

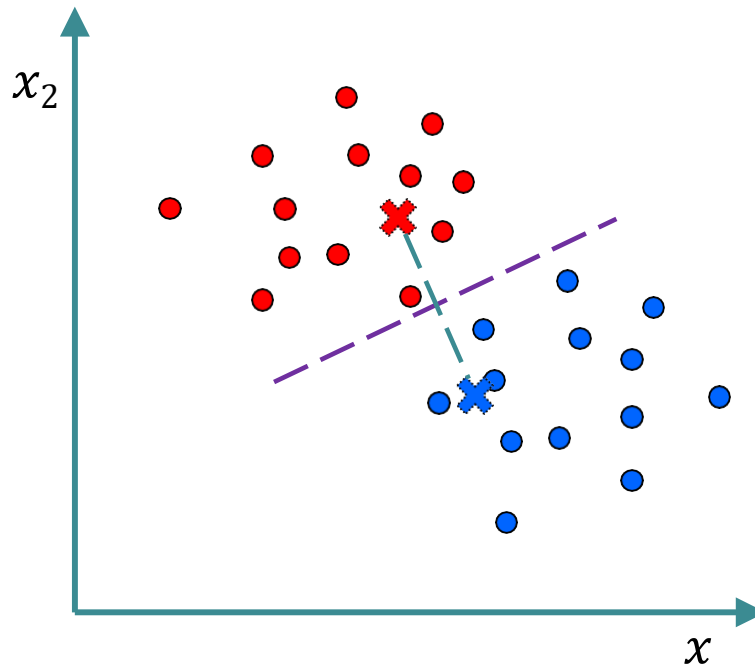
Clustering – K-Means

- Algorithm

- 1º Set the k-number of Cluster
- 2º Choice k points at random, the centroids
- 3º Assign each point to the nearest centroid to create K-clusters.



- 4º Calculate and assign the new centroid of each cluster.
- 5º reassign each point to its nearest centroid. If there have been new assignments go to step 4, otherwise finish.

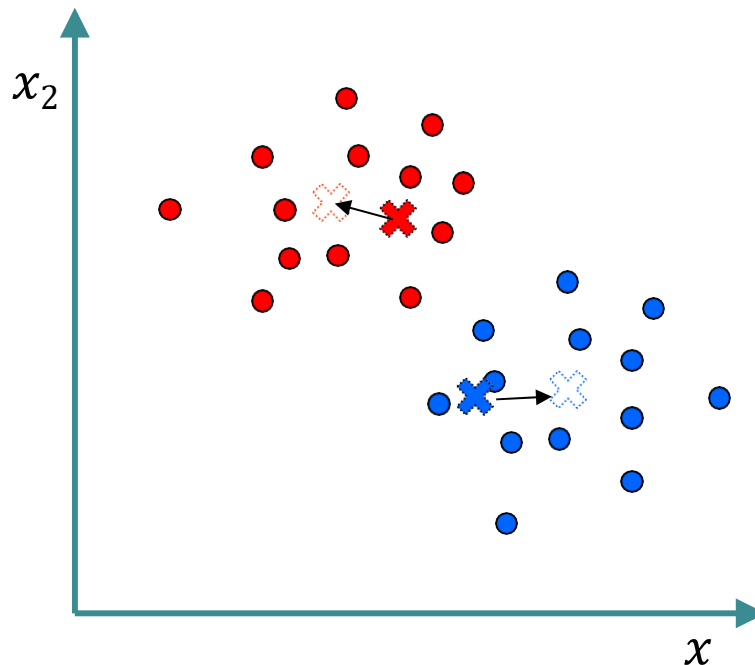


Step 5: reassign each point to its nearest centroid

Clustering – K-Means

- Algorithm

- 1º Set the k-number of Cluster
- 2º Choice k points at random, the centroids
- 3º Assign each point to the nearest centroid to create K-clusters.
- 4º Calculate and assign the new centroid of each cluster.
- 5º reassign each point to its nearest centroid. If there have been new assignments go to step 4, otherwise finish.



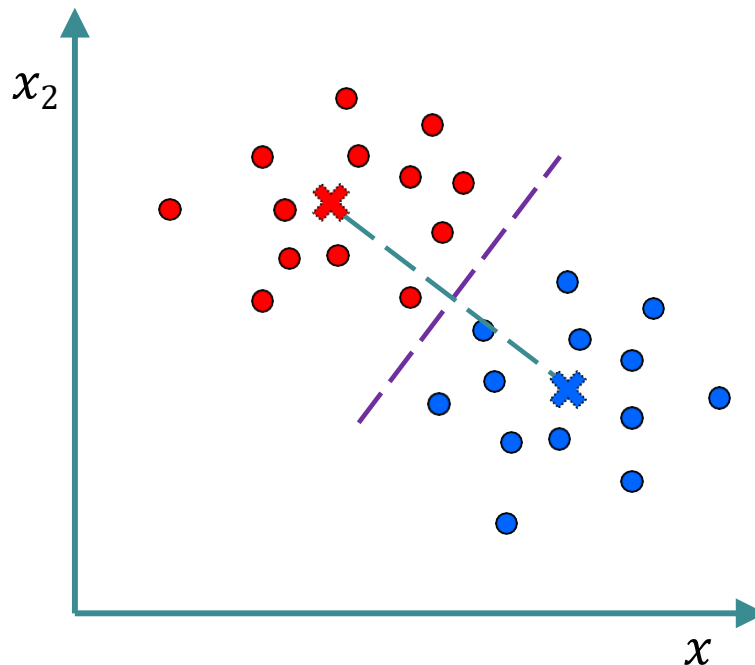
Step 4: Calculate and assign the new centroid

Clustering – K-Means

- Algorithm

- 1º Set the k-number of Cluster
- 2º Choice k points at random, the centroids
- 3º Assign each point to the nearest centroid to create K-clusters.

- 4º Calculate and assign the new centroid of each cluster.
- 5º reassign each point to its nearest centroid. If there have been new assignments go to step 4, otherwise finish.



Step 5: reassign each point to its nearest centroid

Clustering – K-Means

Algorithm

1° Set the k-number of Cluster

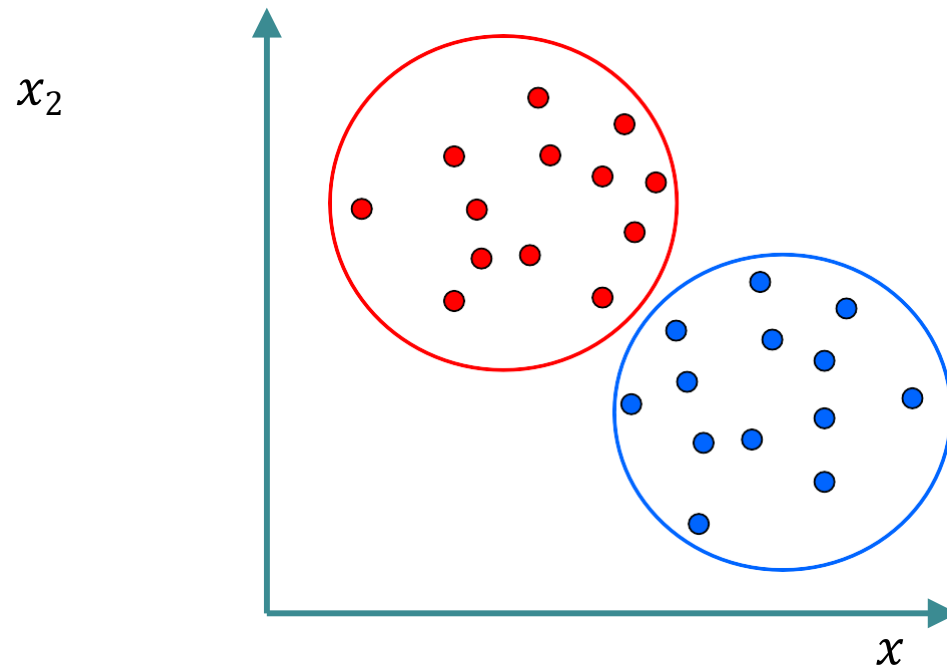
2° Choice k points at random, the centroids

3° Assign each point to the nearest centroid to create K-clusters.



4° Calculate and assign the new centroid of each cluster.

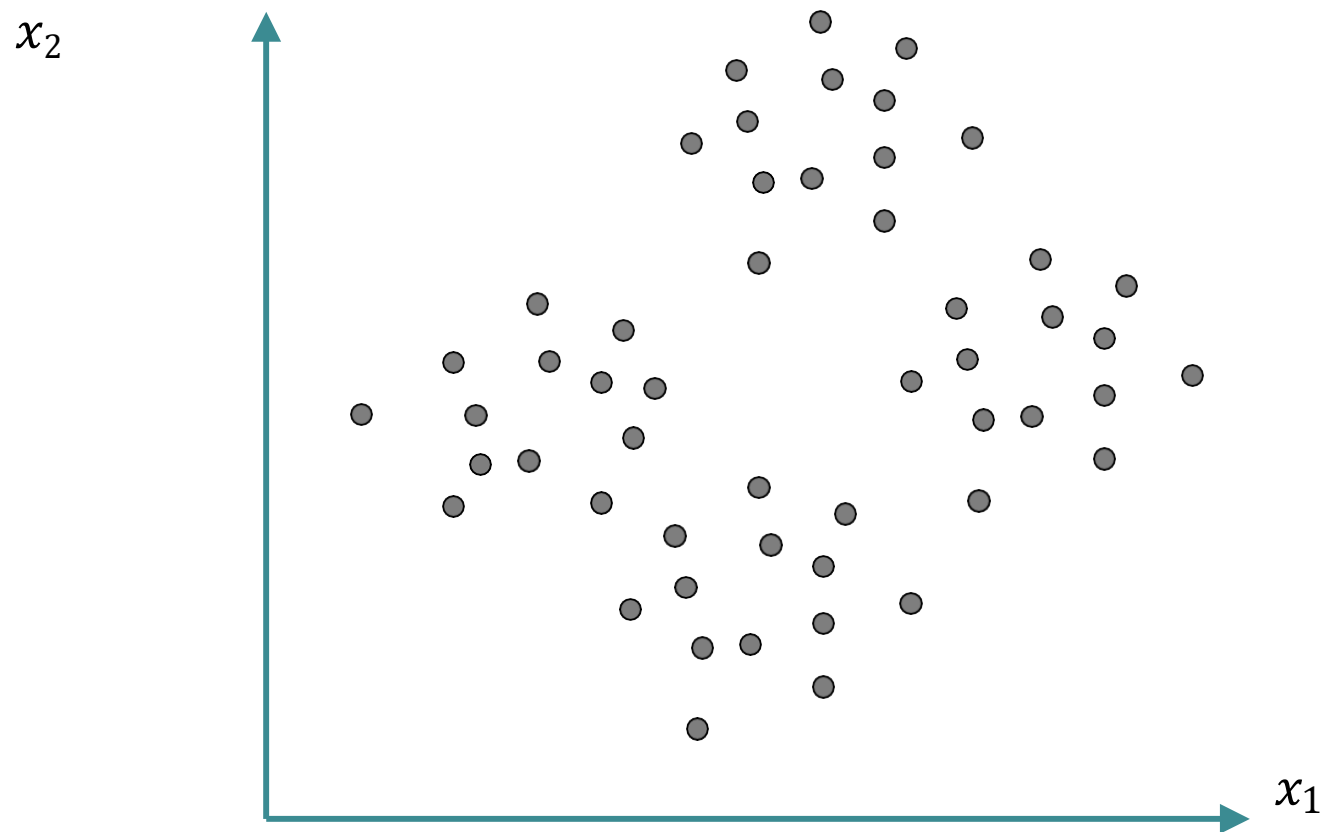
5° reassign each point to its nearest centroid. If there have been new assignments go to step 4, otherwise finish.



Clustering – K-Means

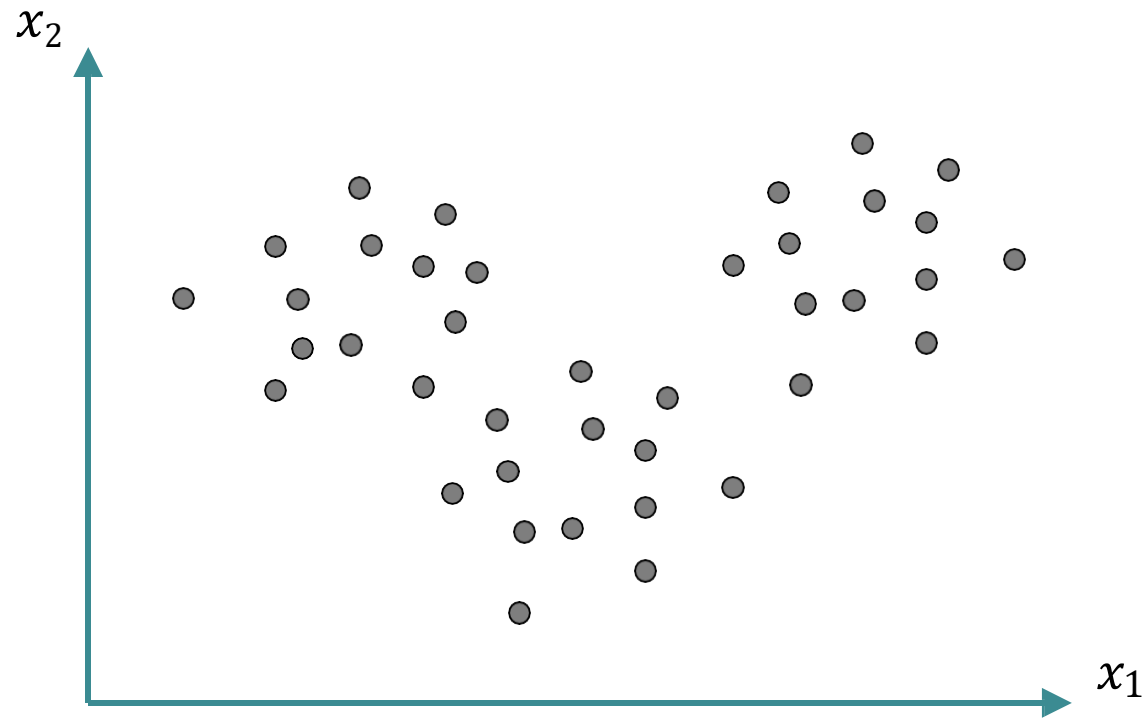
The Problem of Random Initialization of centroids.

K=3



Clustering – K-Means

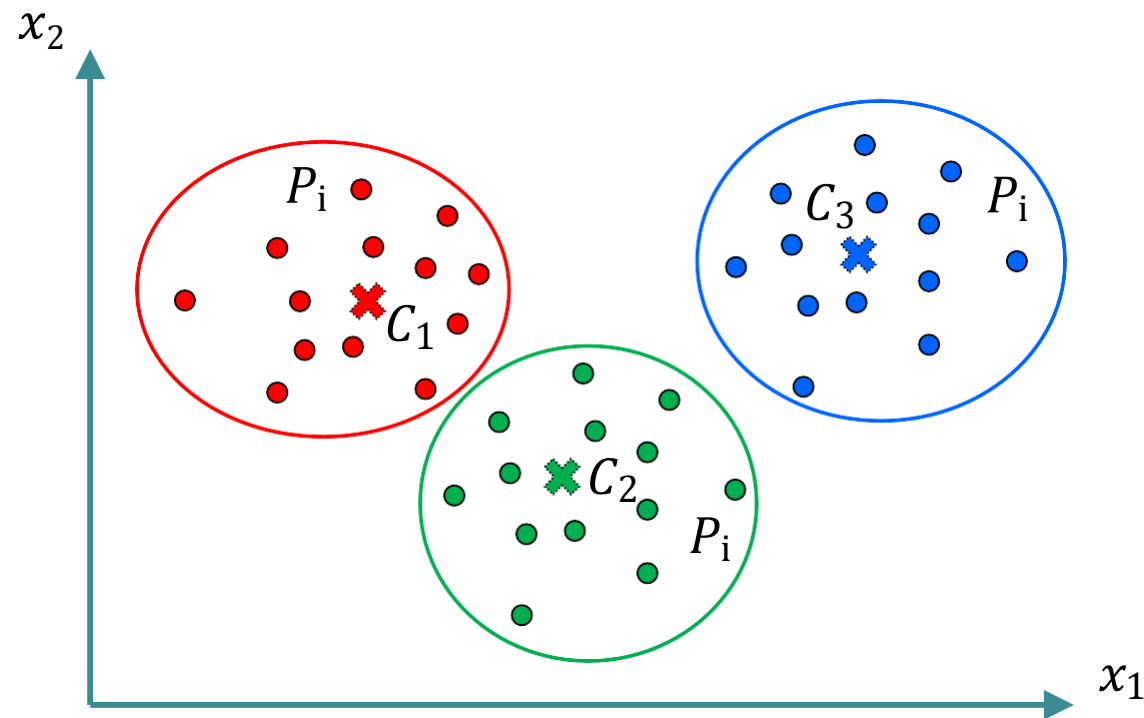
How to choose the right number of clusters.



Clustering – K-Means

How to choose the right number of clusters.

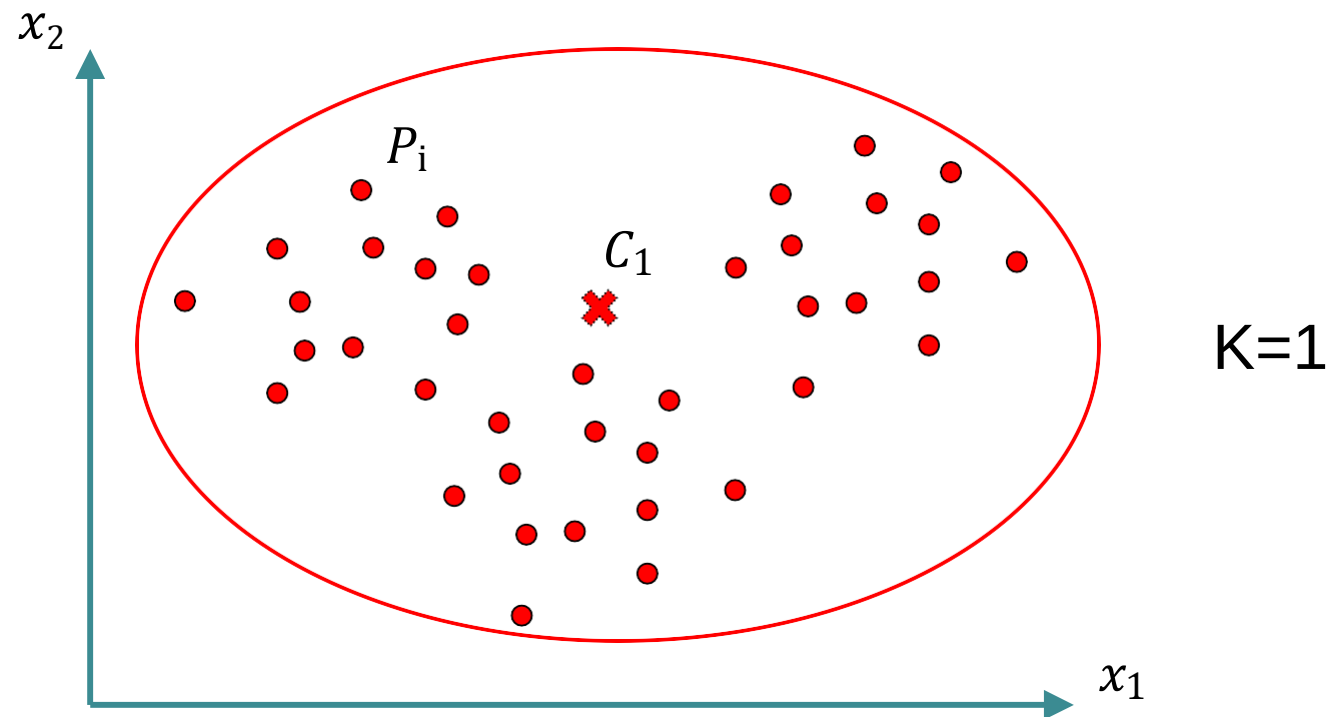
$$WCSS = \sum_{P_i \in \text{Cluster1}} d(P_i, C_1)^2 + \sum_{P_i \in \text{Cluster2}} d(P_i, C_2)^2 + \sum_{P_i \in \text{Cluster3}} d(P_i, C_3)^2$$



Clustering – K-Means

How to choose the right number of clusters.

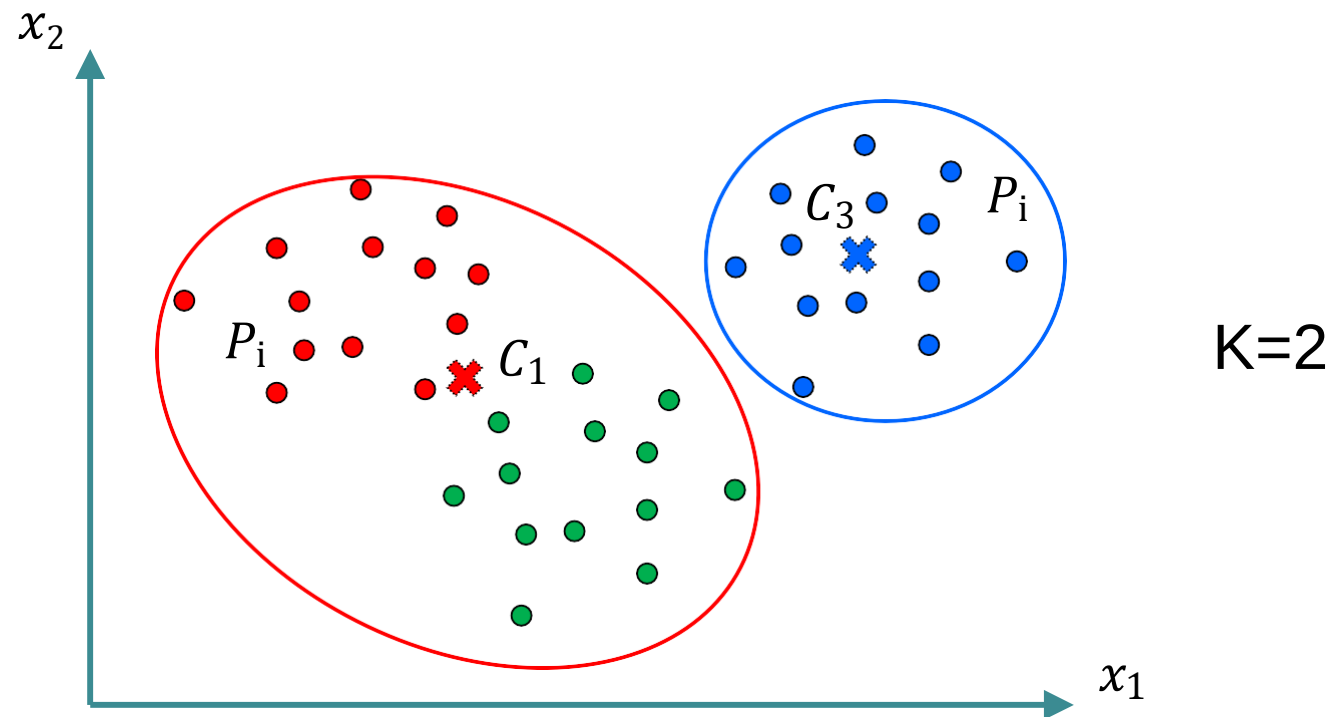
$$WCSS = \sum_{P_i \in \text{Cluster1}} d(P_i, C_1)^2$$



Clustering – K-Means

How to choose the right number of clusters.

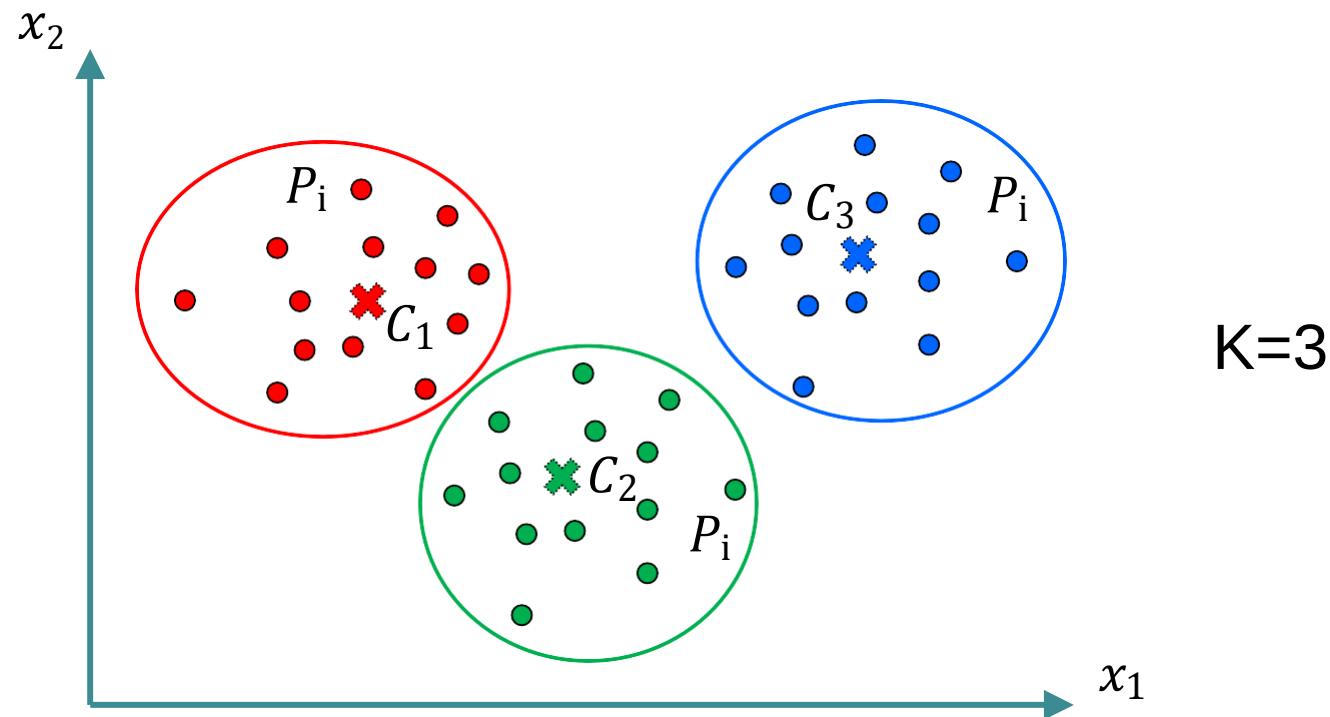
$$WCSS = \sum_{P_i \in \text{Cluster1}} d(P_i, C_1)^2 + \sum_{P_i \in \text{Cluster2}} d(P_i, C_2)^2$$



Clustering – K-Means

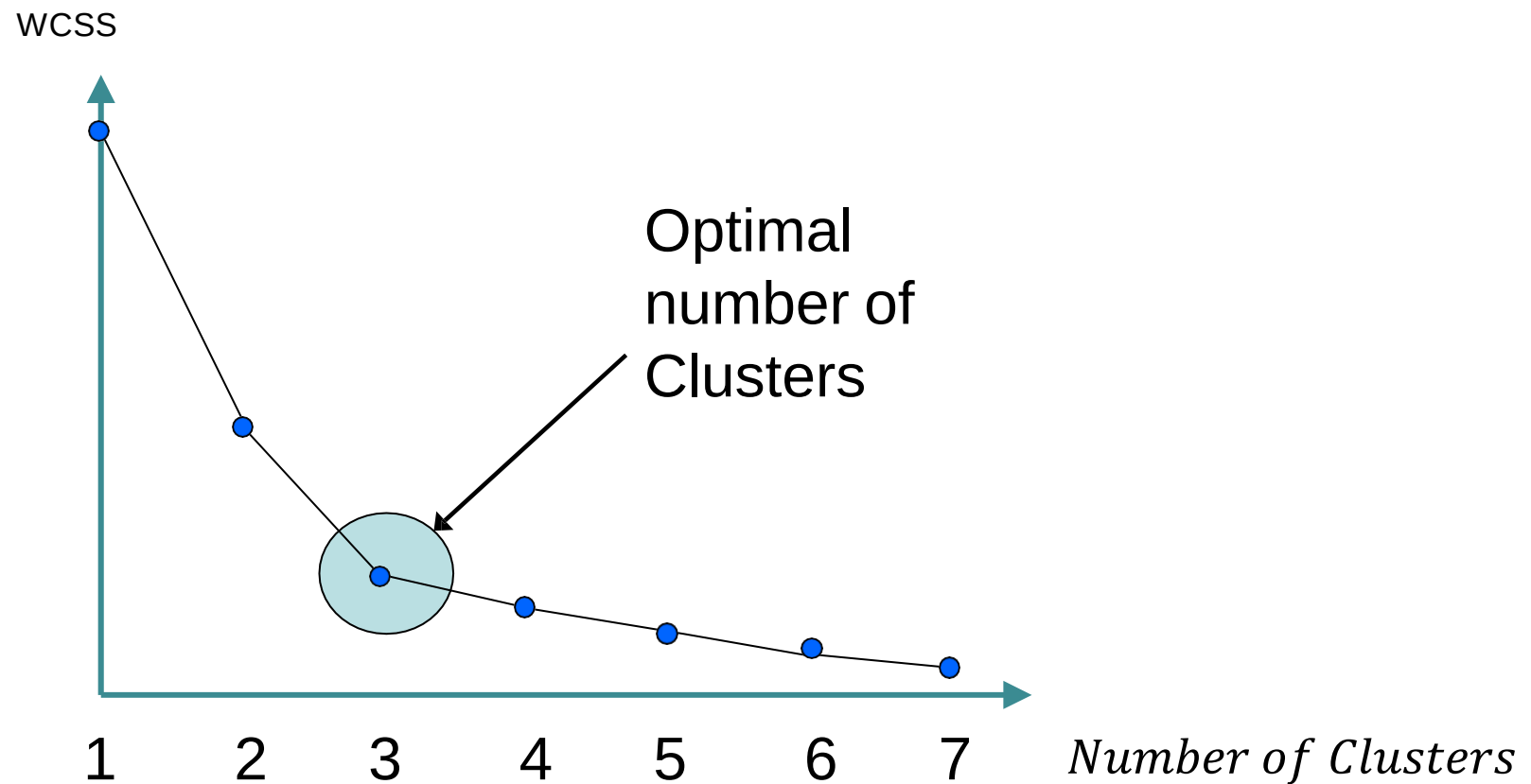
How to choose the right number of clusters.

$$WCSS = \sum_{P_i \in \text{Cluster1}} d(P_i, C_1)^2 + \sum_{P_i \in \text{Cluster2}} d(P_i, C_2)^2 + \sum_{P_i \in \text{Cluster3}} d(P_i, C_3)^2$$



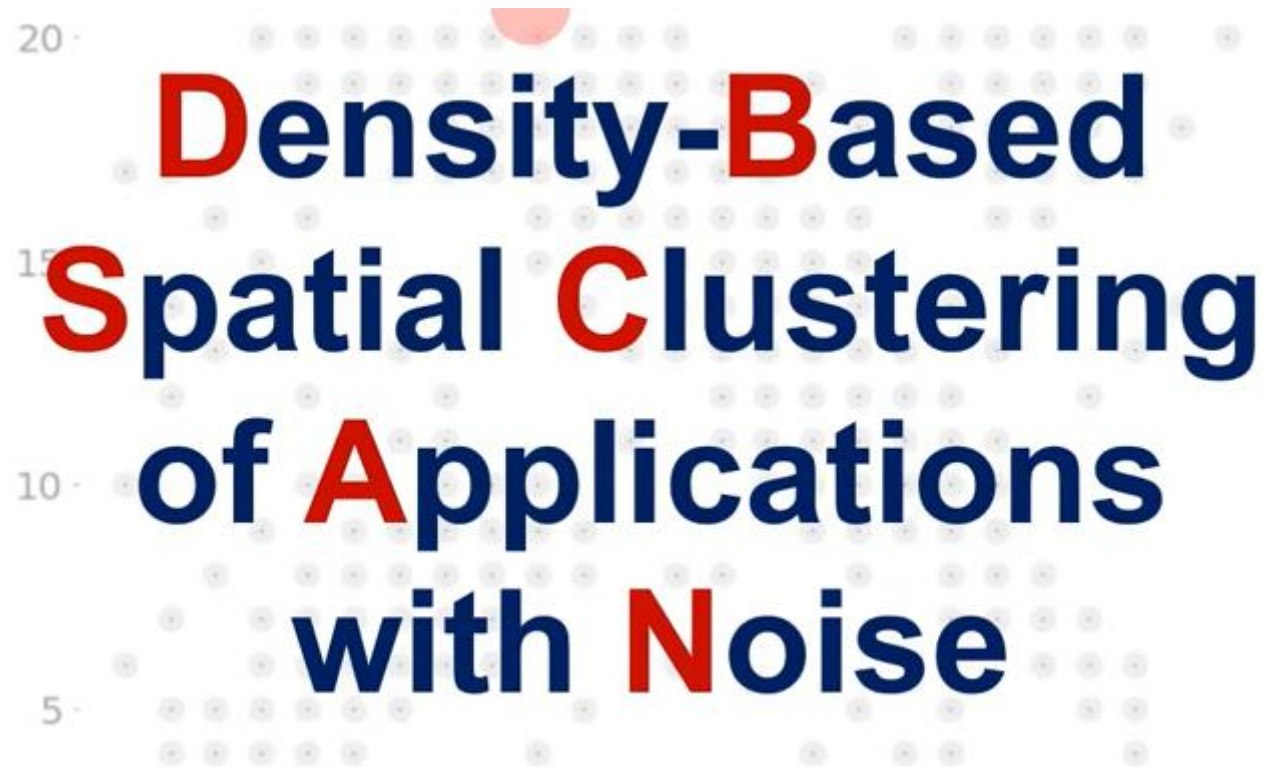
Clustering – K-Means

How to choose the right number of clusters.



Clustering – DBSCAN

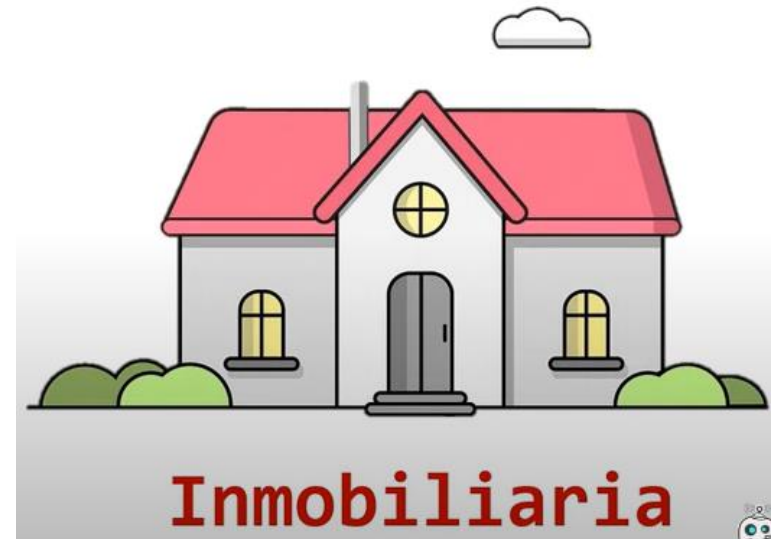
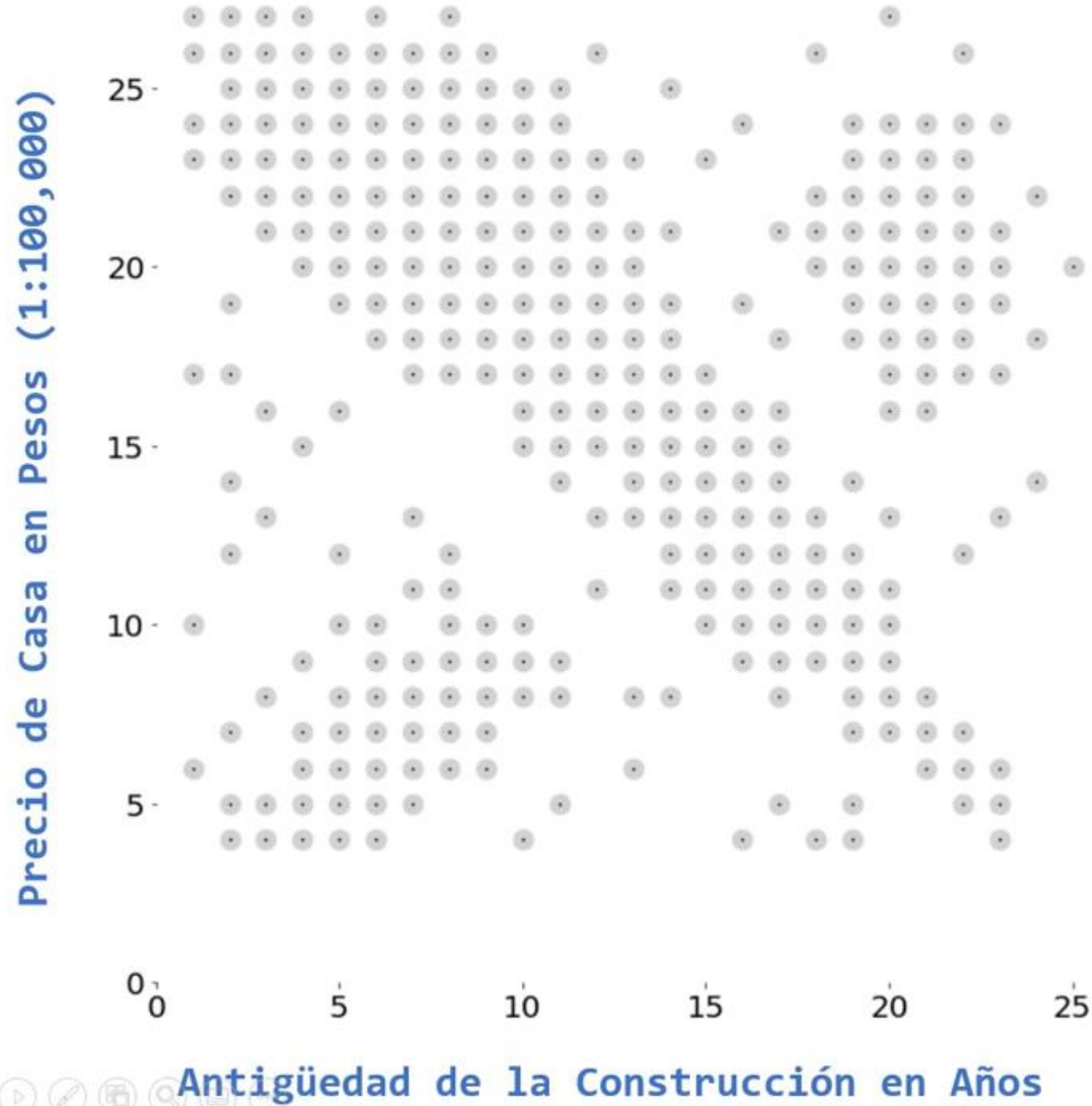
Clustering – DBSCAN



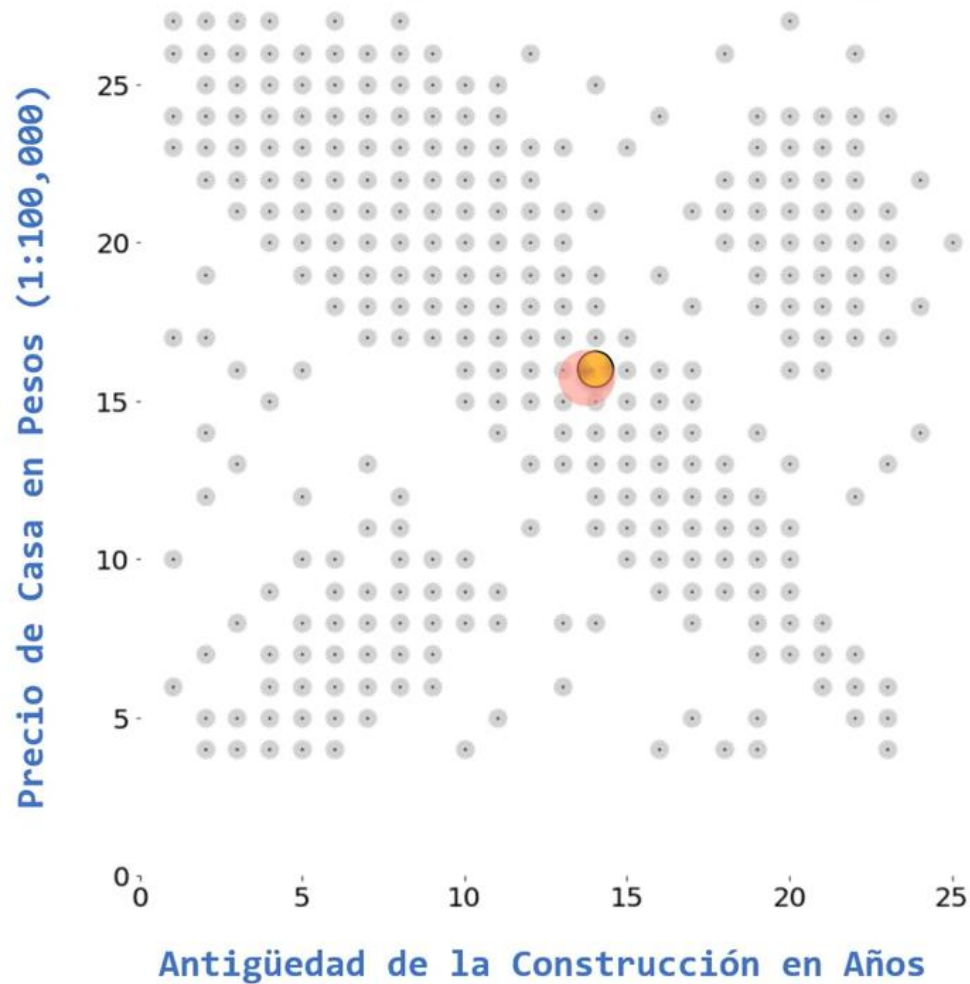
**Density-Based
Spatial Clustering
of Applications
with Noise**

DBSCAN

Clustering – DBSCAN

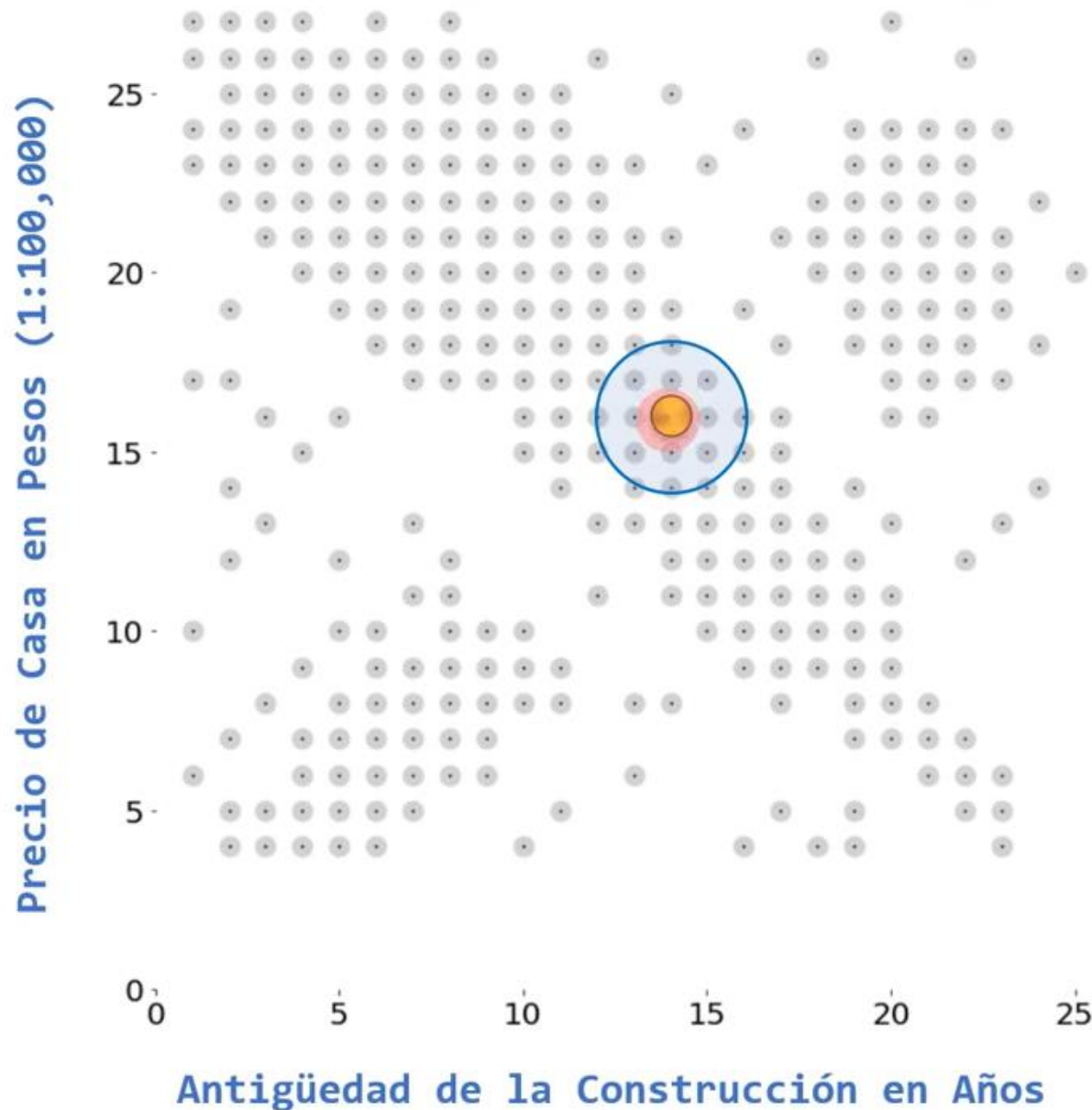


Clustering – DBSCAN



**Selección de
cualquier
punto**

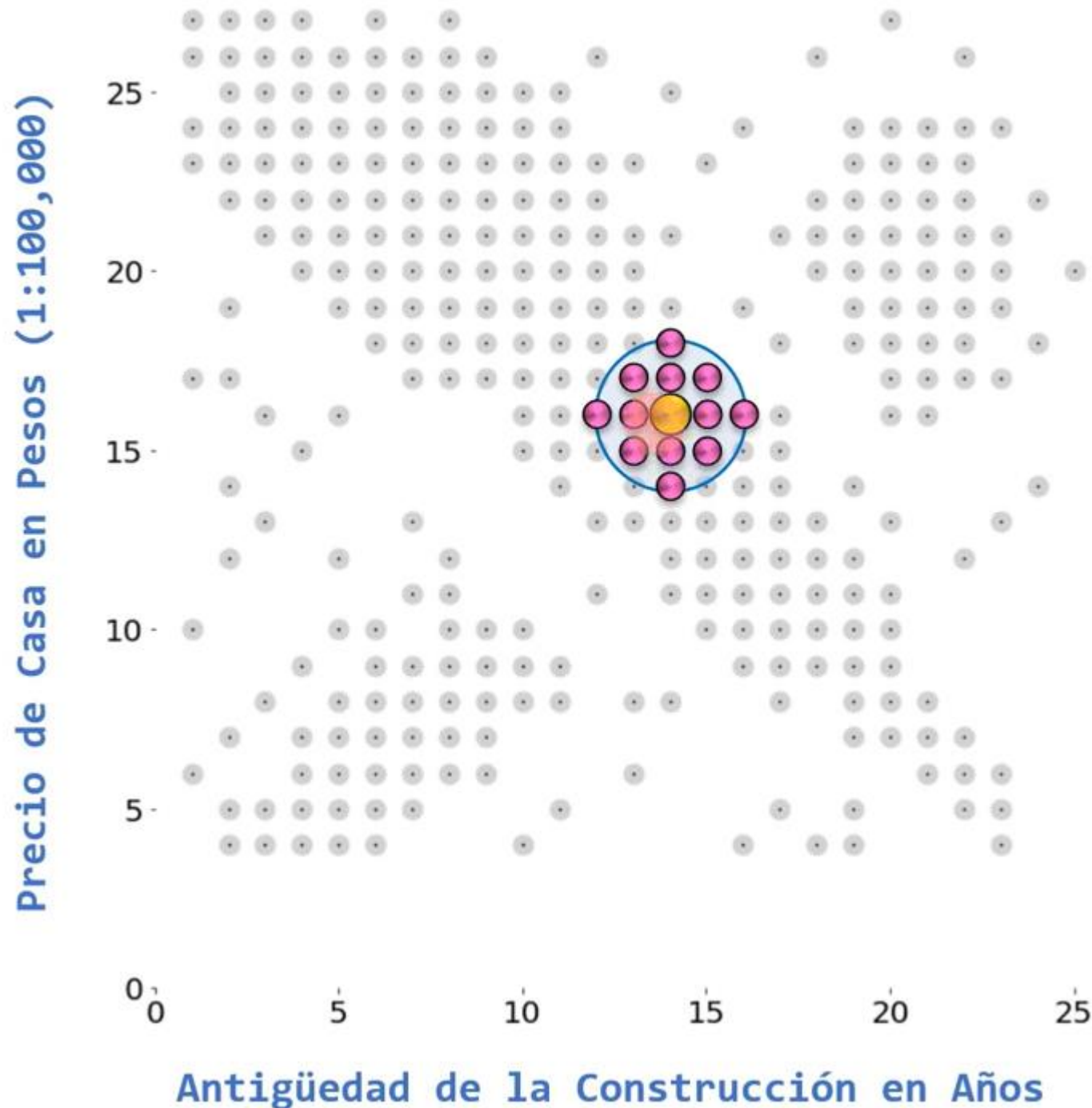
Clustering – DBSCAN



épsilon (ϵ) = 2

**Distancia que
marca los límites
del Vecindario**

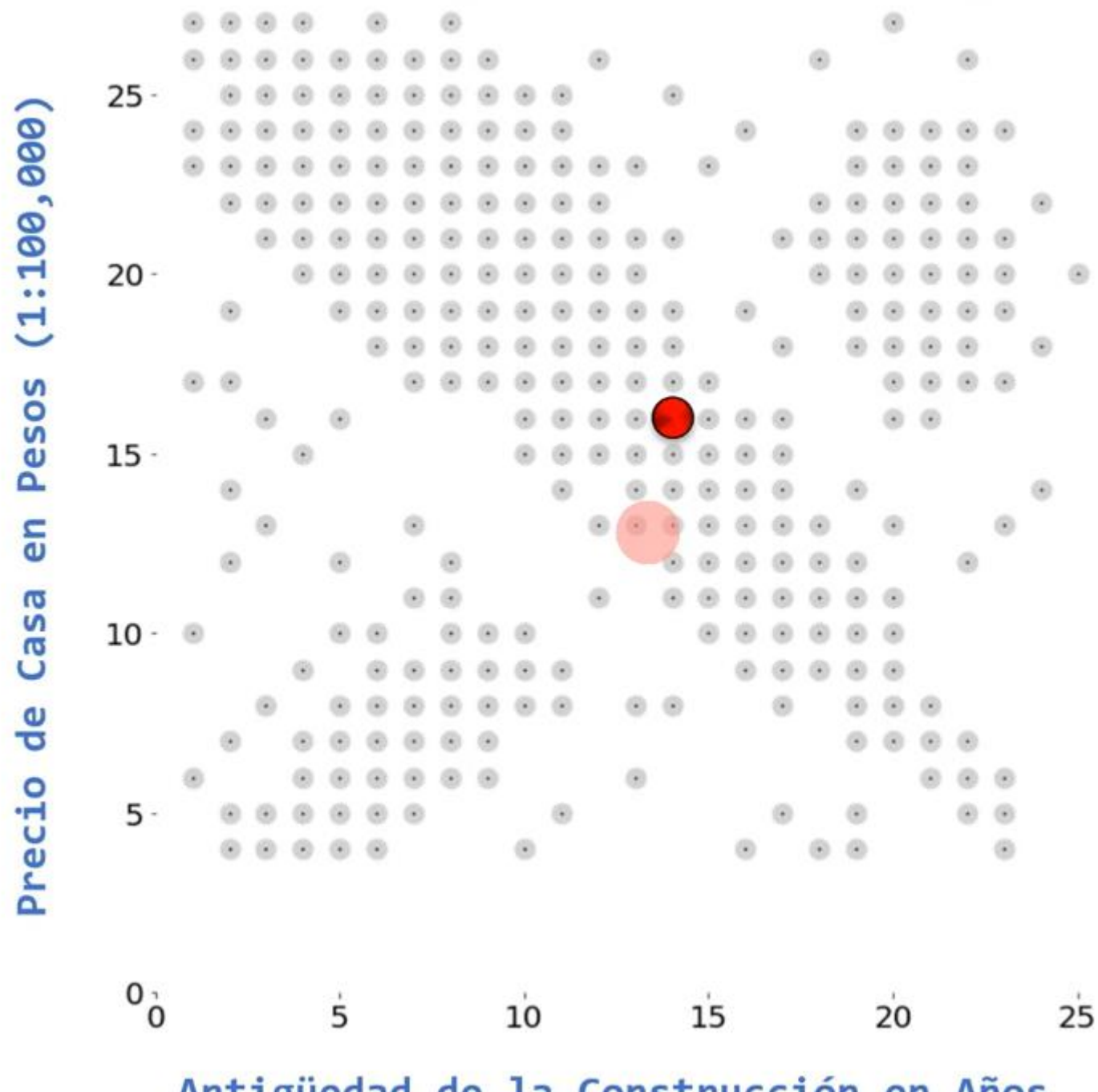
Clustering – DBSCAN



puntos = 10

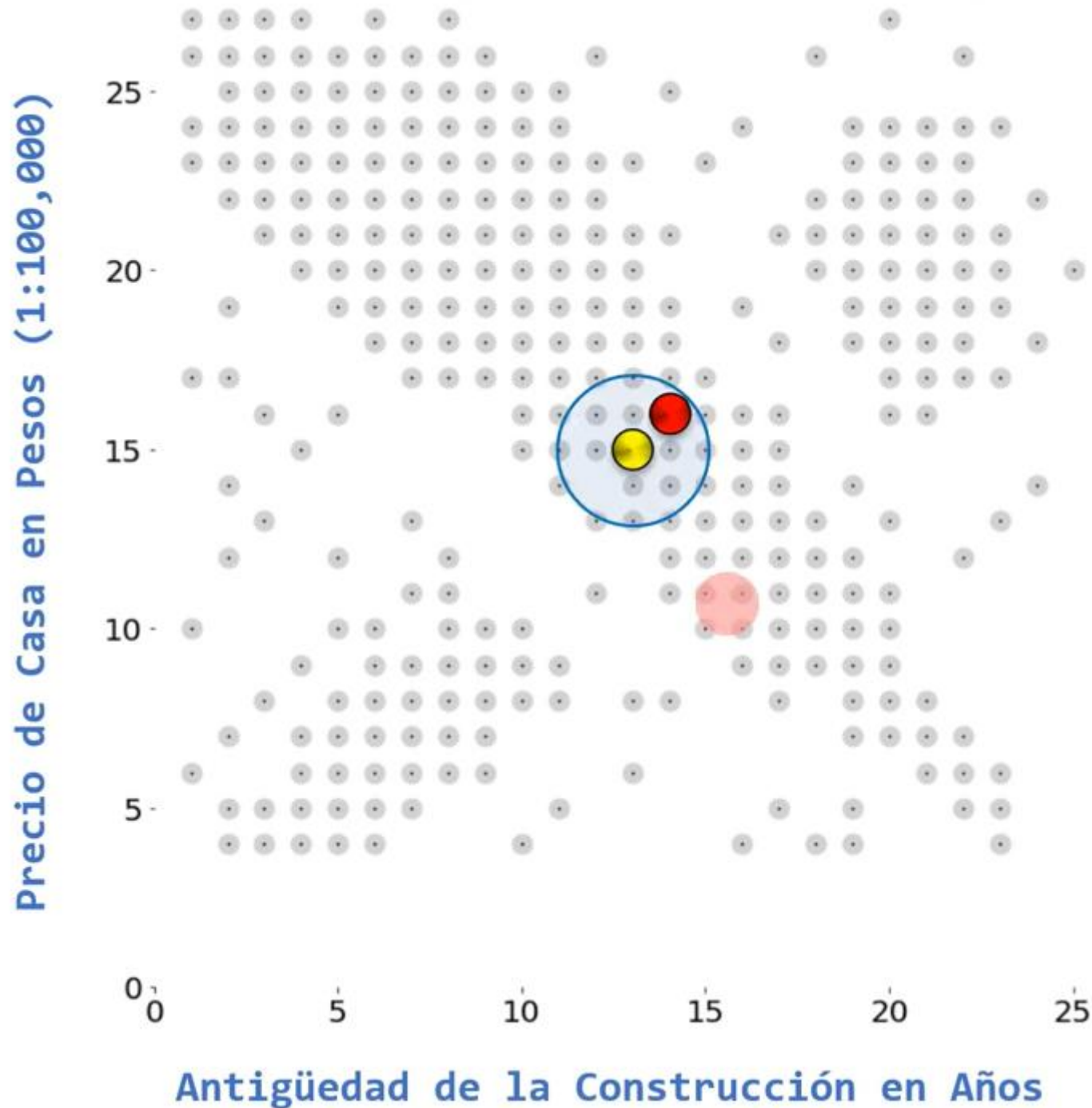
Densidad mínima
del Vecindario

Clustering – DBSCAN



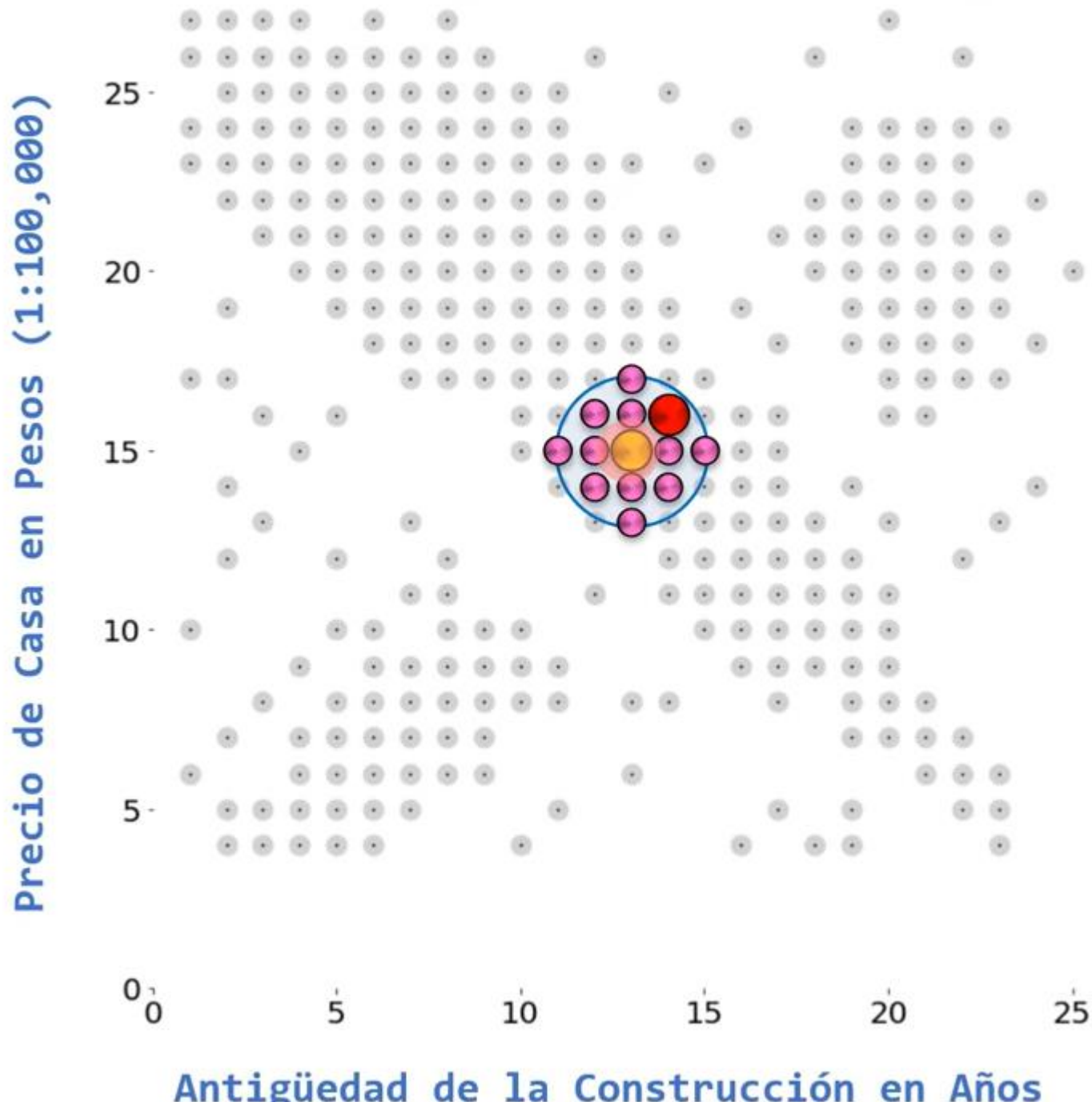
Si hay densidad,
entonces es
punto CORE
(central)

Clustering – DBSCAN



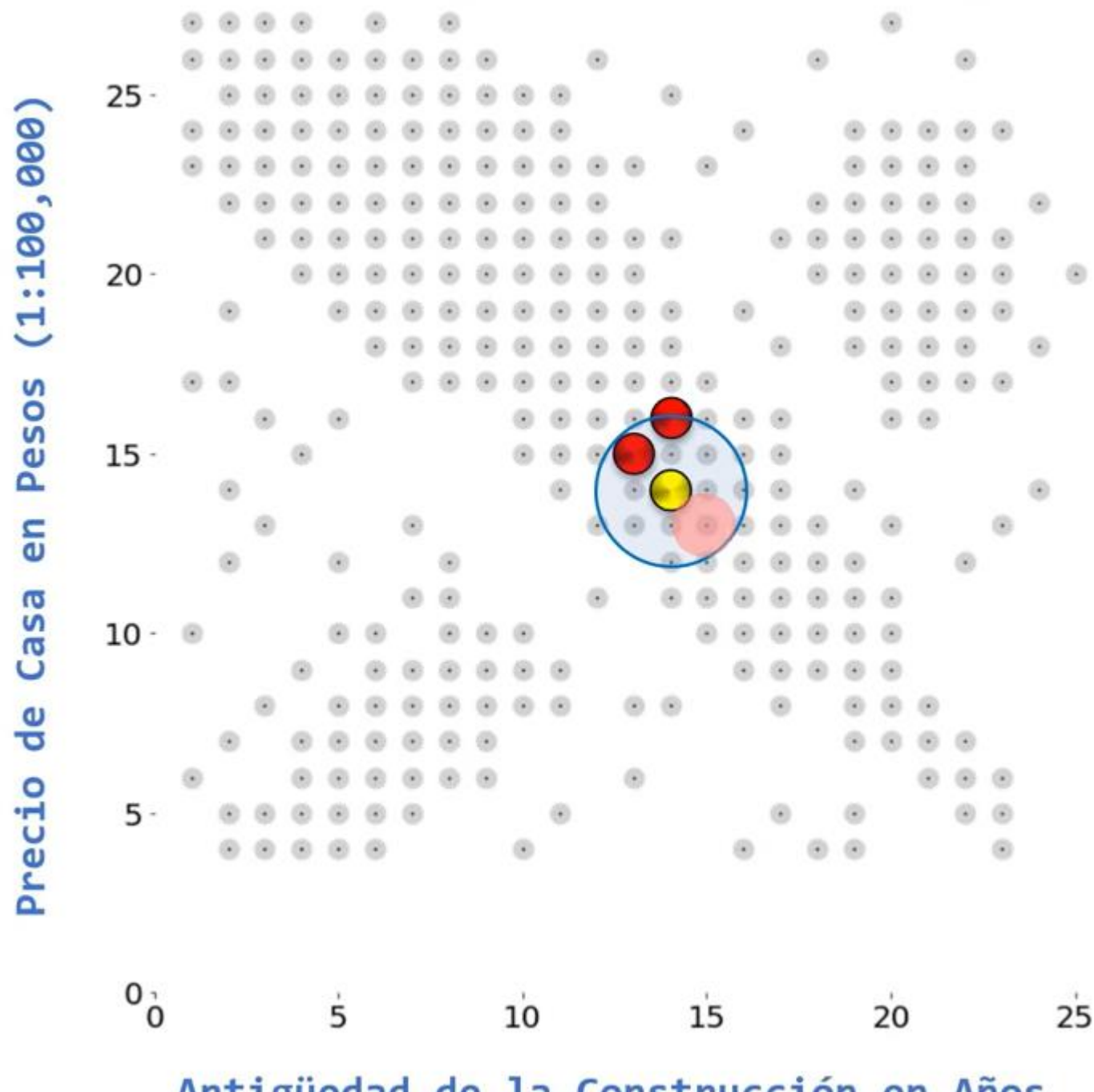
Se toma el siguiente punto para determinar su vecindad y densidad

Clustering – DBSCAN



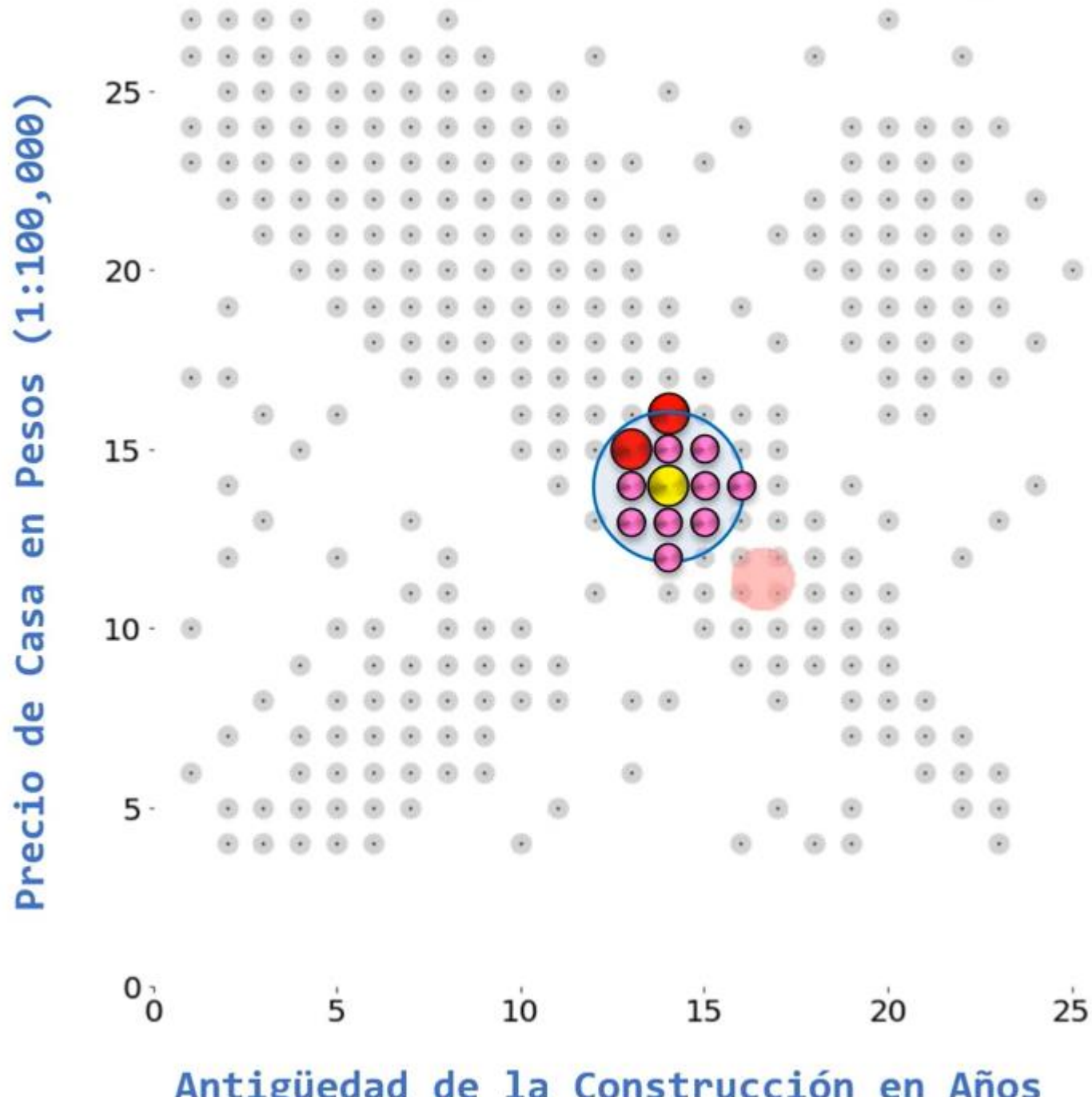
**Si hay densidad,
entonces es
punto CORE
(central)**

Clustering – DBSCAN



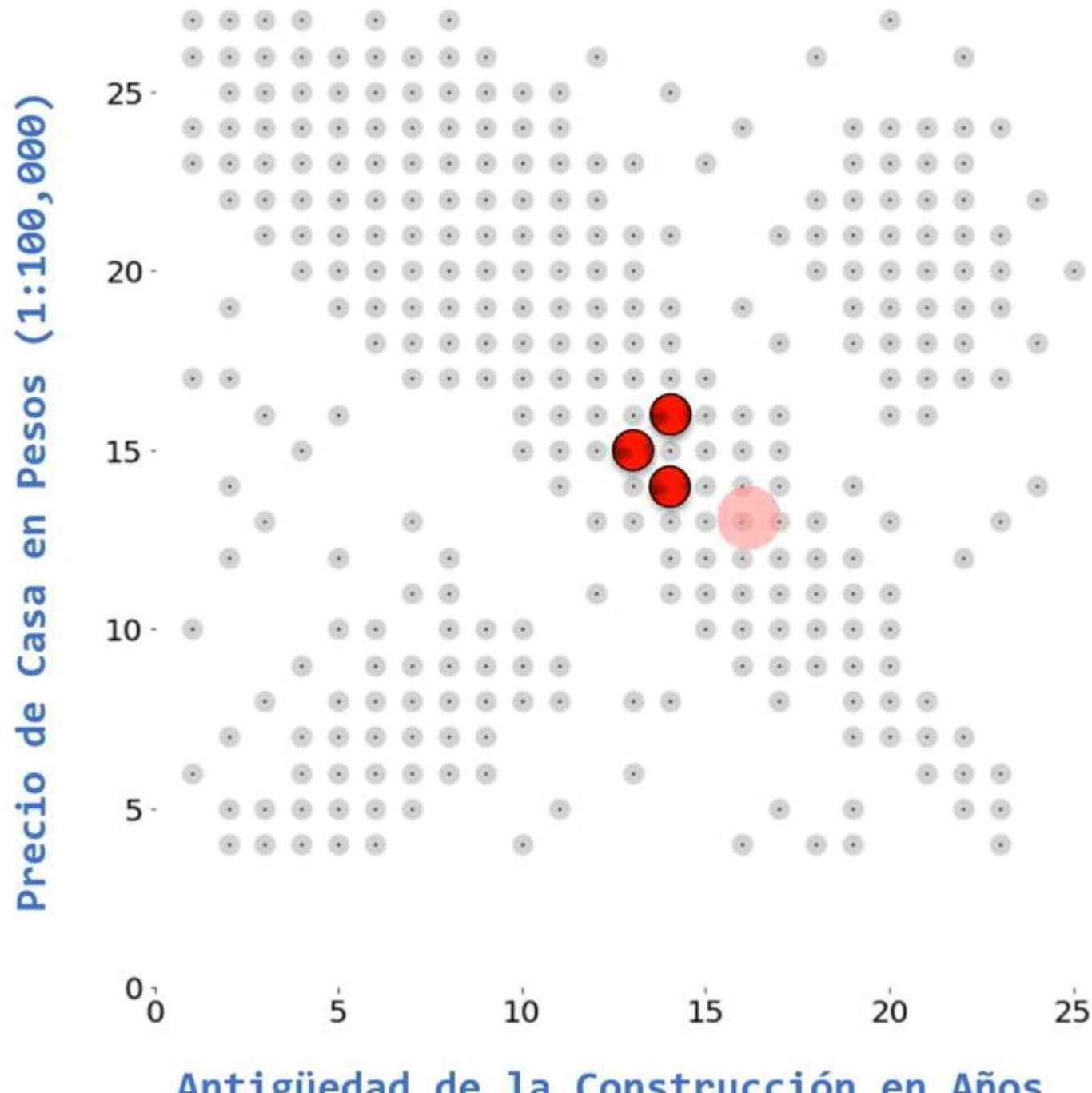
Se toma el
siguiente punto

Clustering – DBSCAN



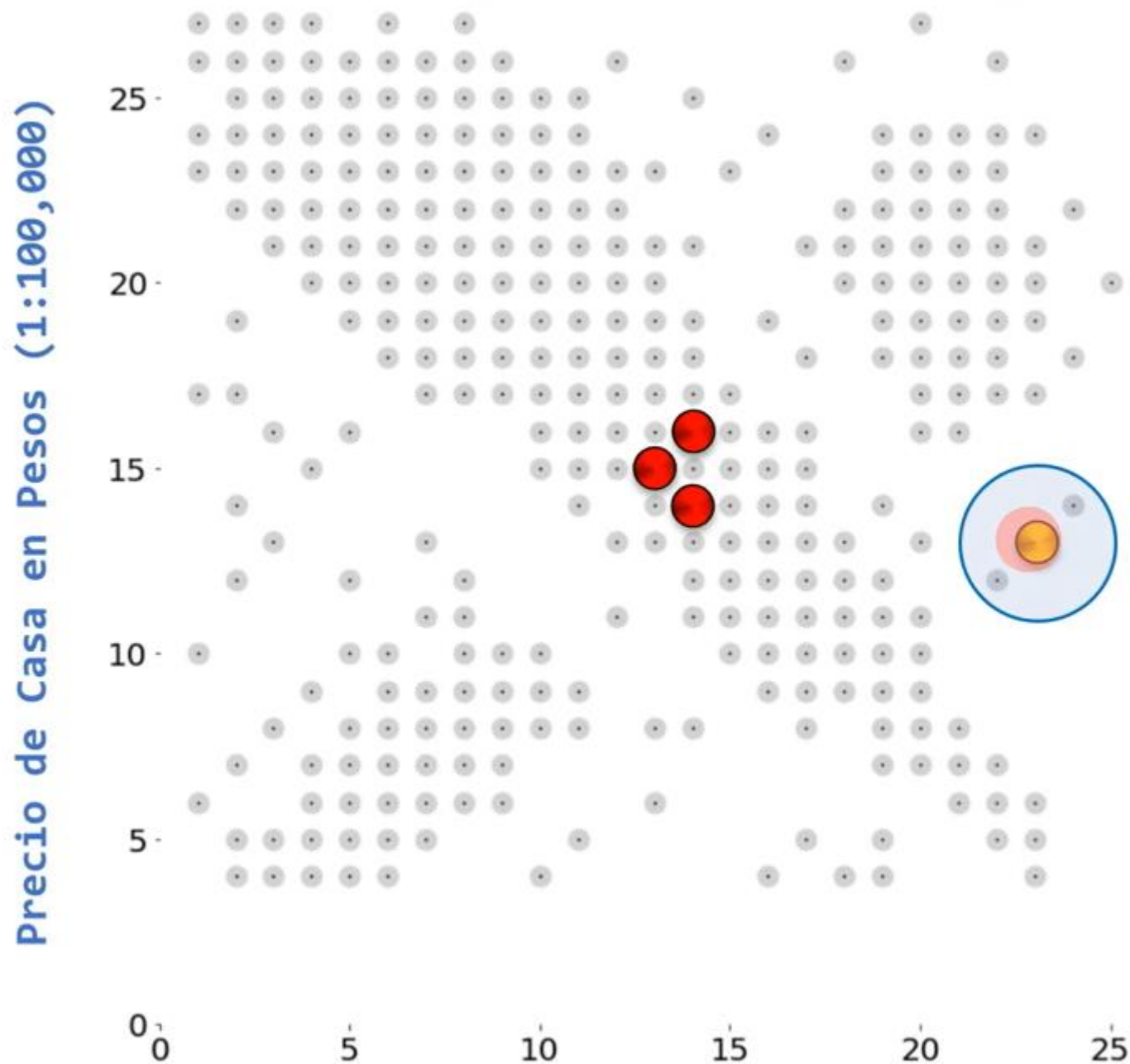
Si hay densidad,
entonces es
punto CORE
(central)

Clustering – DBSCAN



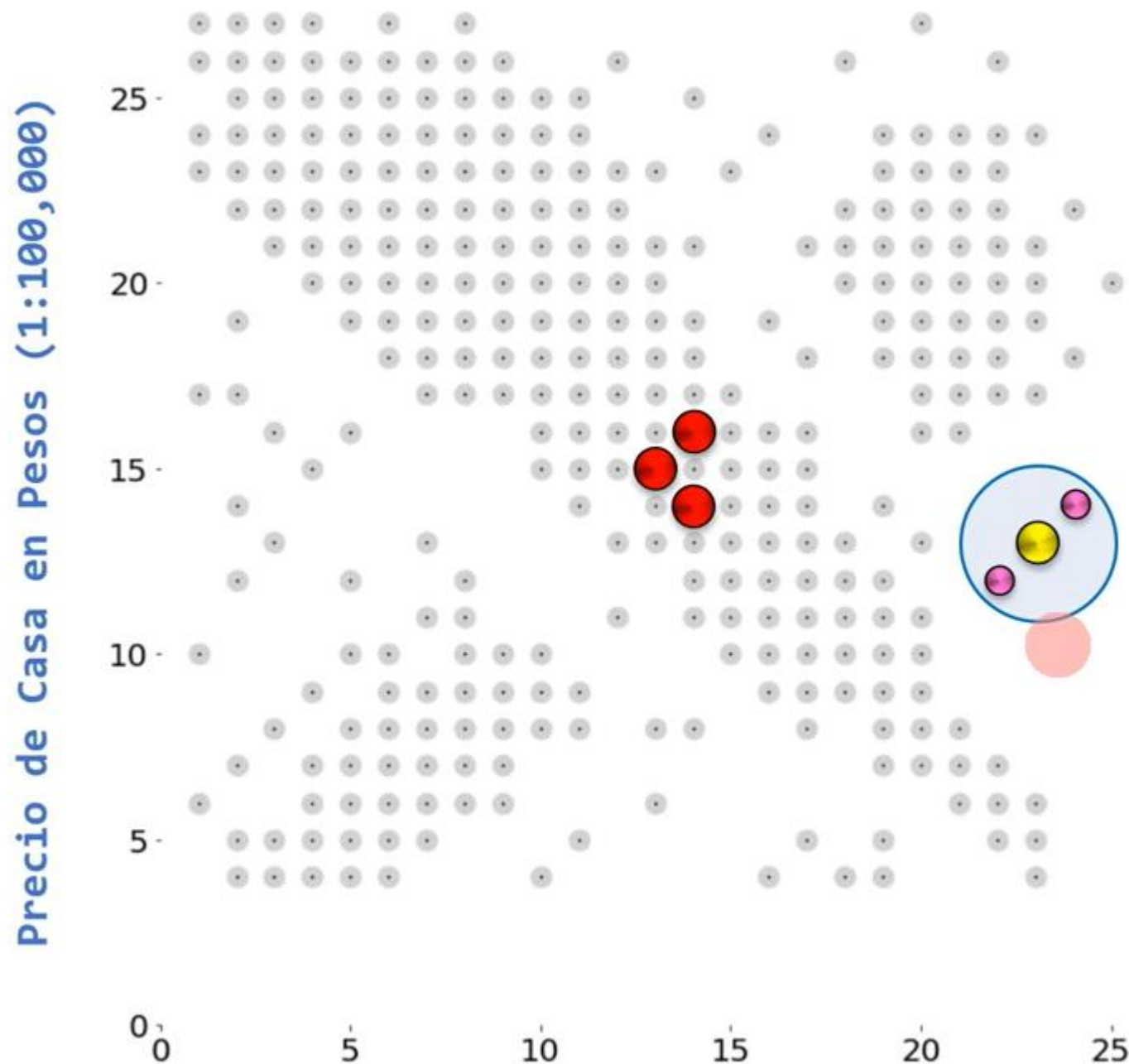
Si hay densidad,
entonces es
punto CORE
(central)

Clustering – DBSCAN



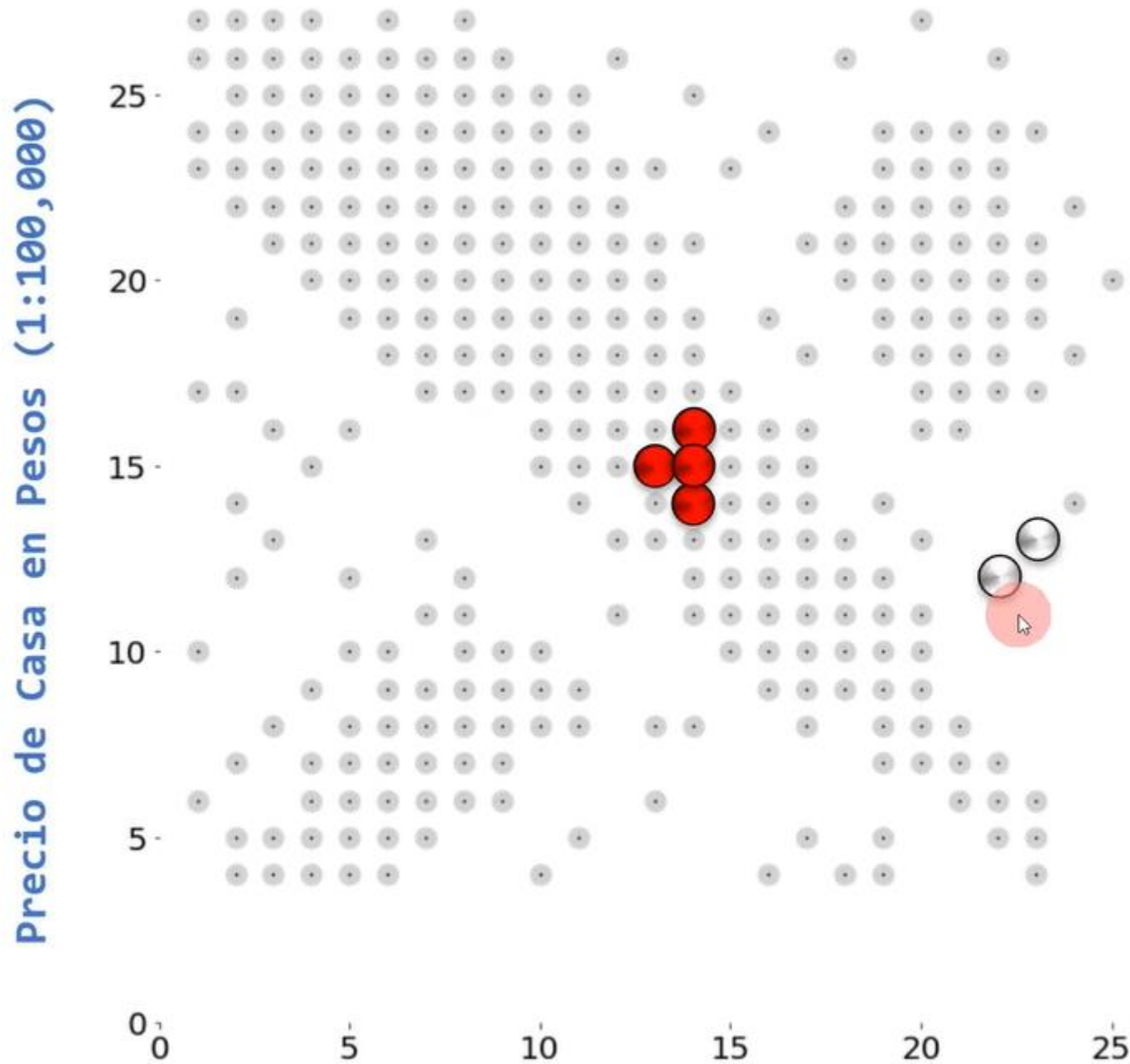
**Se toma el
siguiente punto
arbitrariamente**

Clustering – DBSCAN



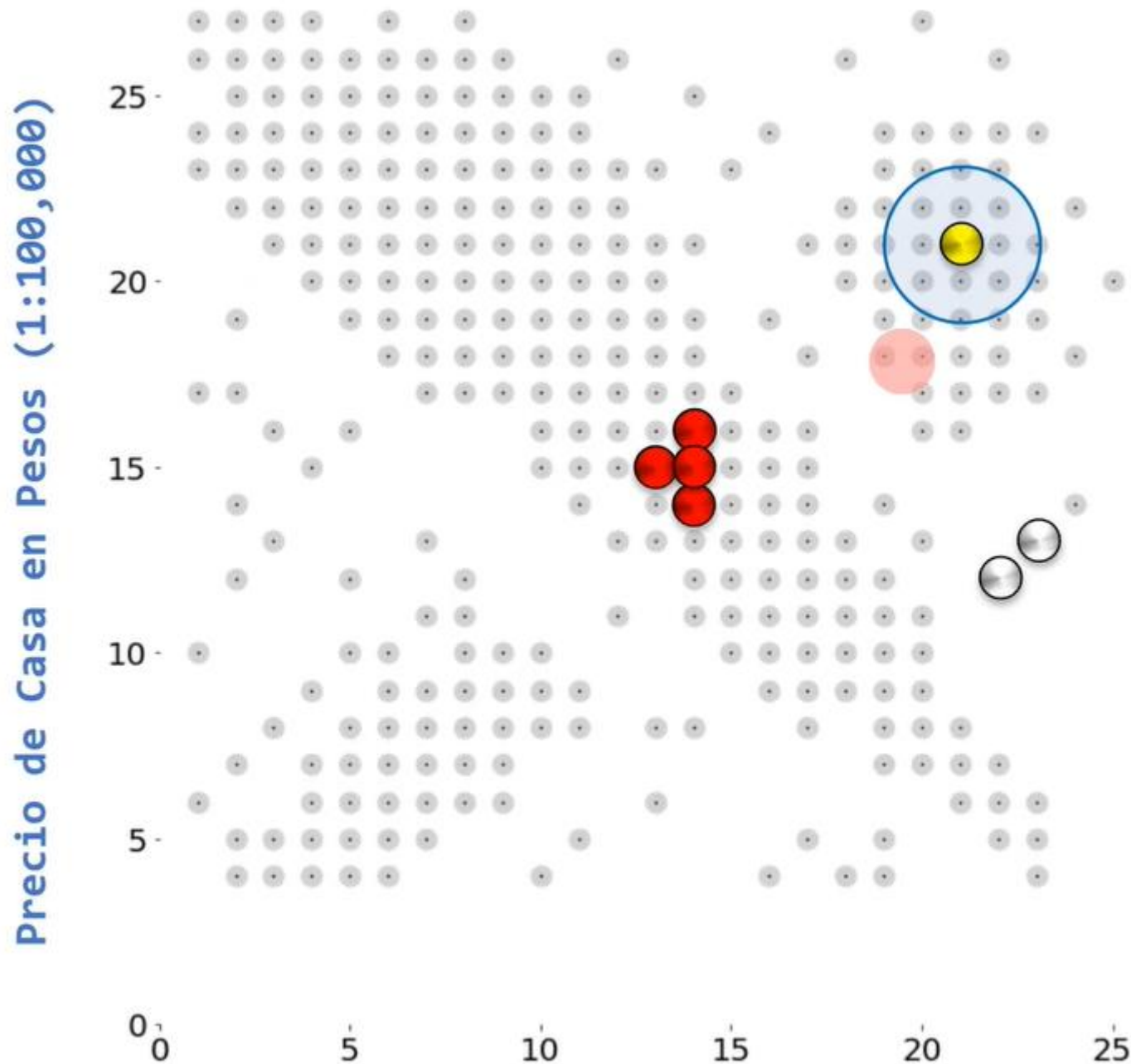
**Si NO hay
densidad,
entonces
RUIDO**

Clustering – DBSCAN



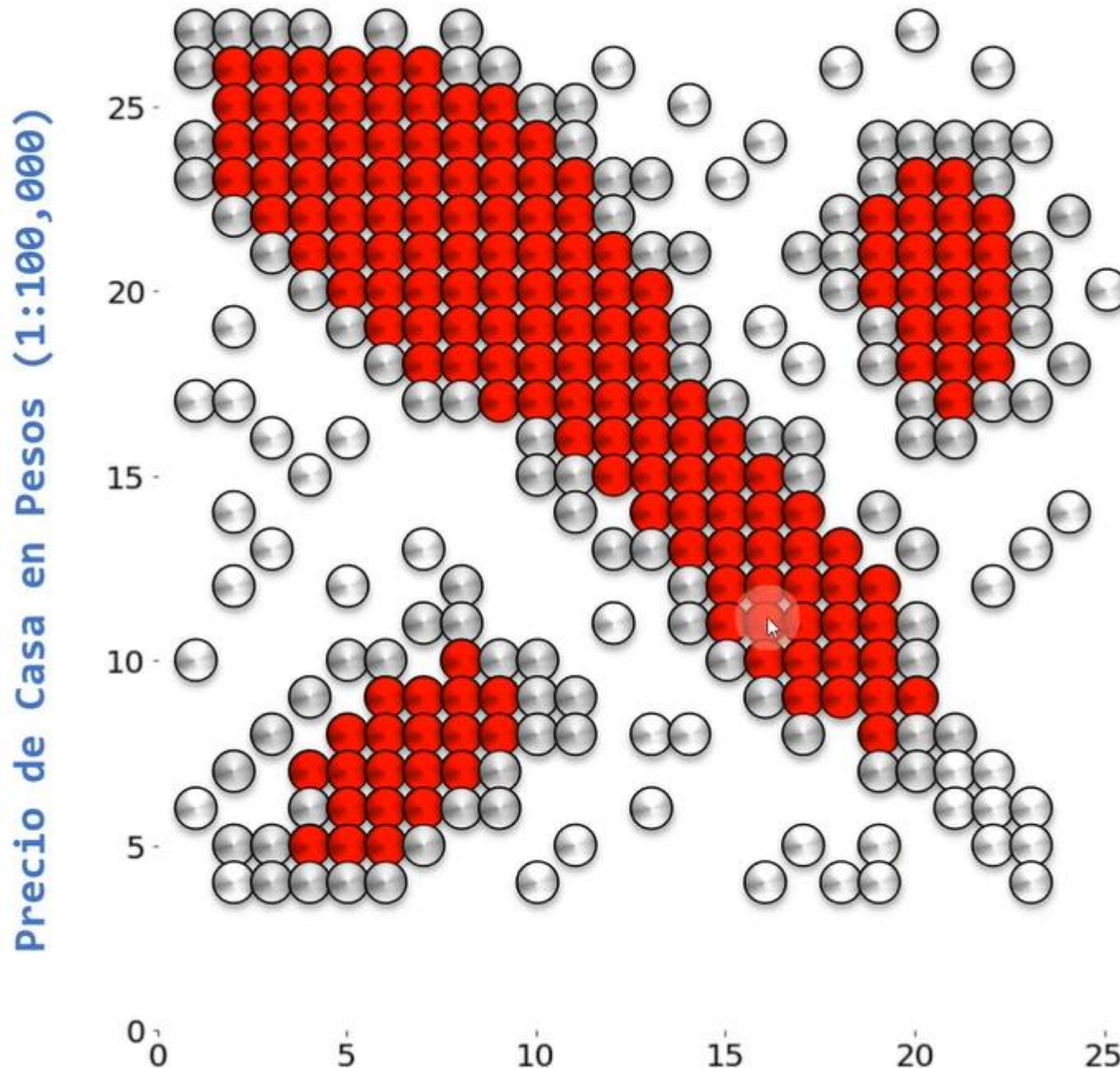
Todo punto se
etiqueta como
CORE o **RUIDO**

Clustering – DBSCAN



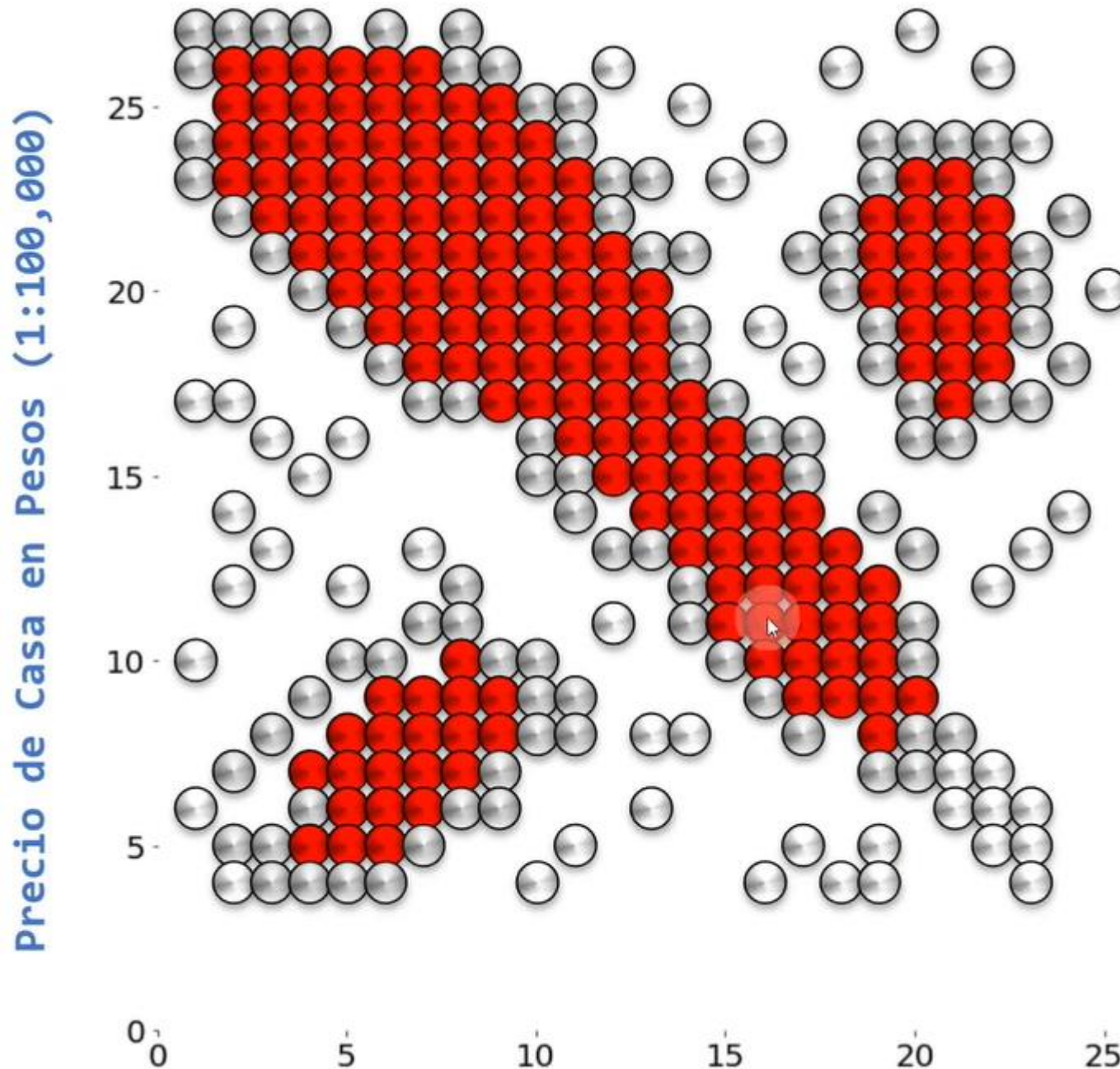
Se toma
el siguiente, y
el siguiente, y
el siguiente ...

Clustering – DBSCAN



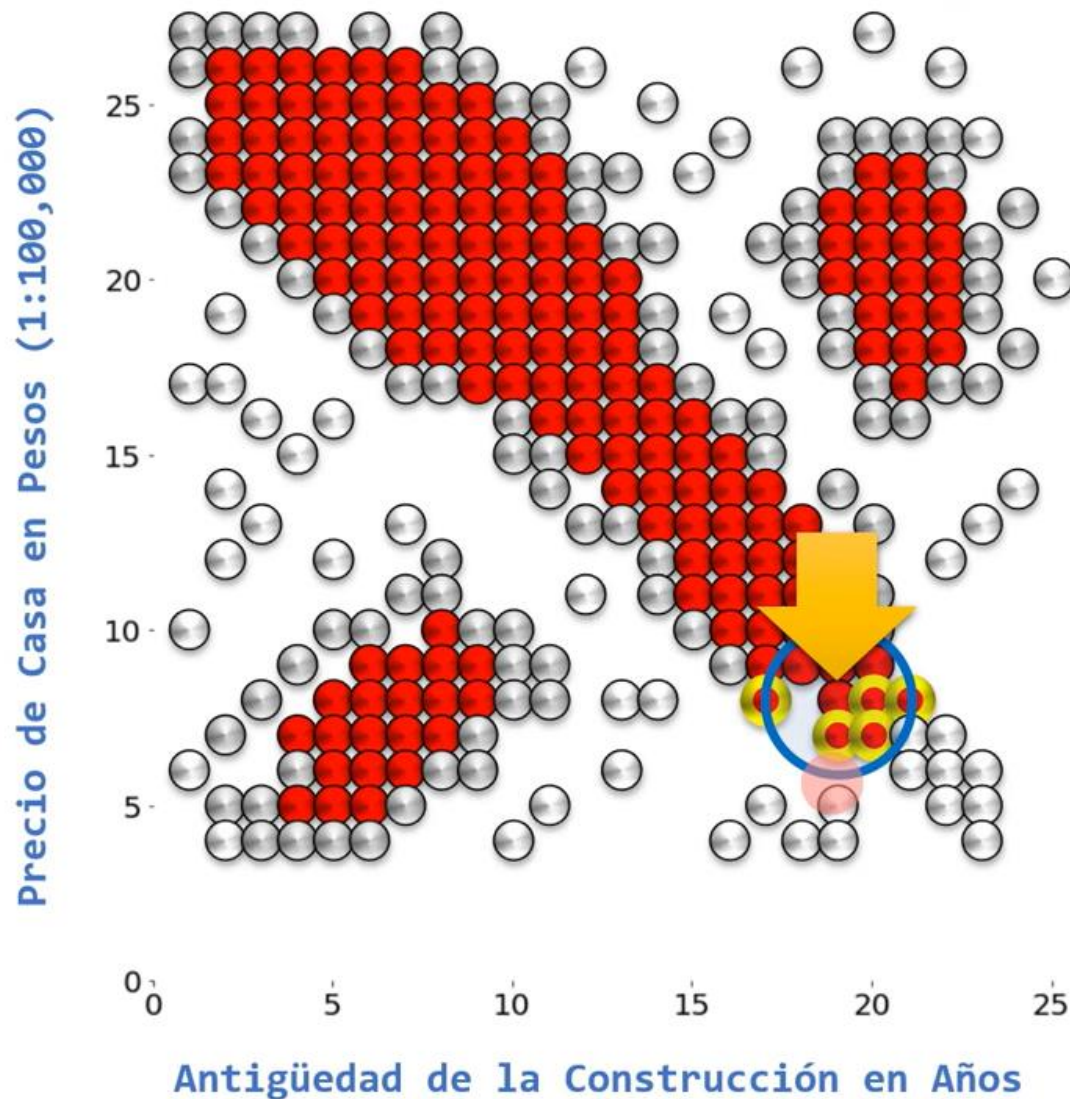
Resultado:
los puntos
han sido
etiquetados

Clustering – DBSCAN



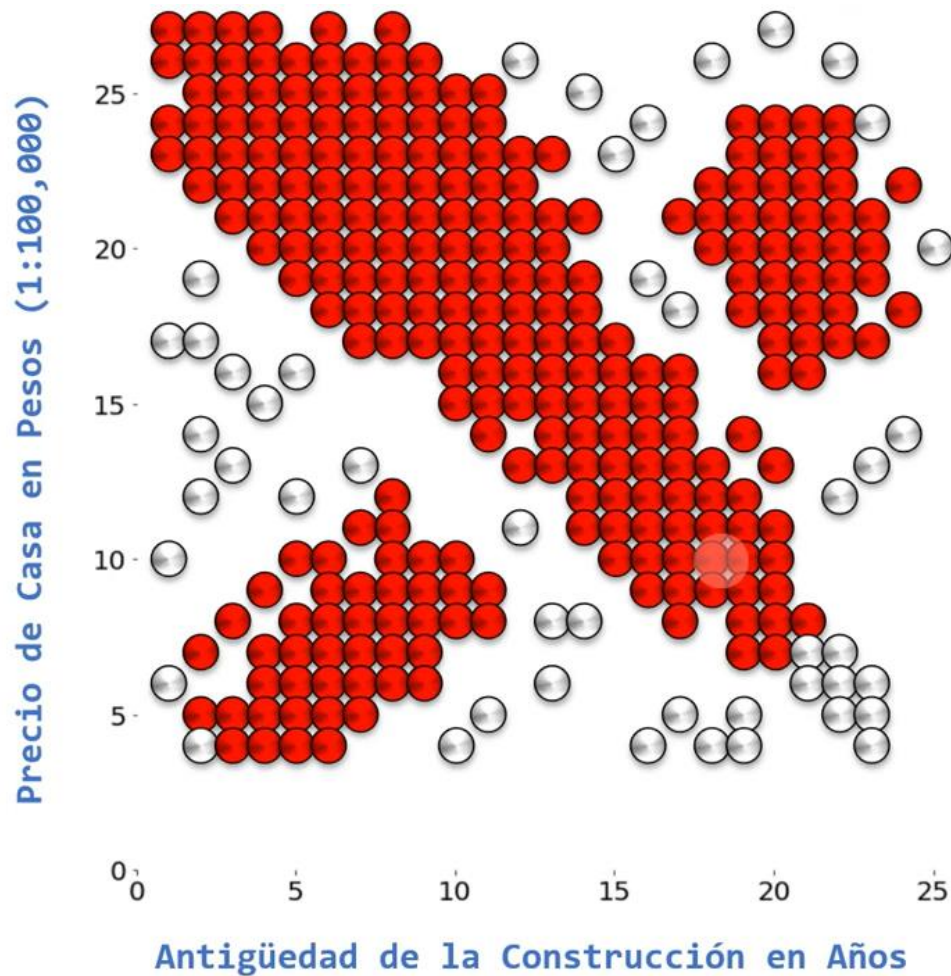
**¿Hay puntos
contiguos
a los CORES?**

Clustering – DBSCAN



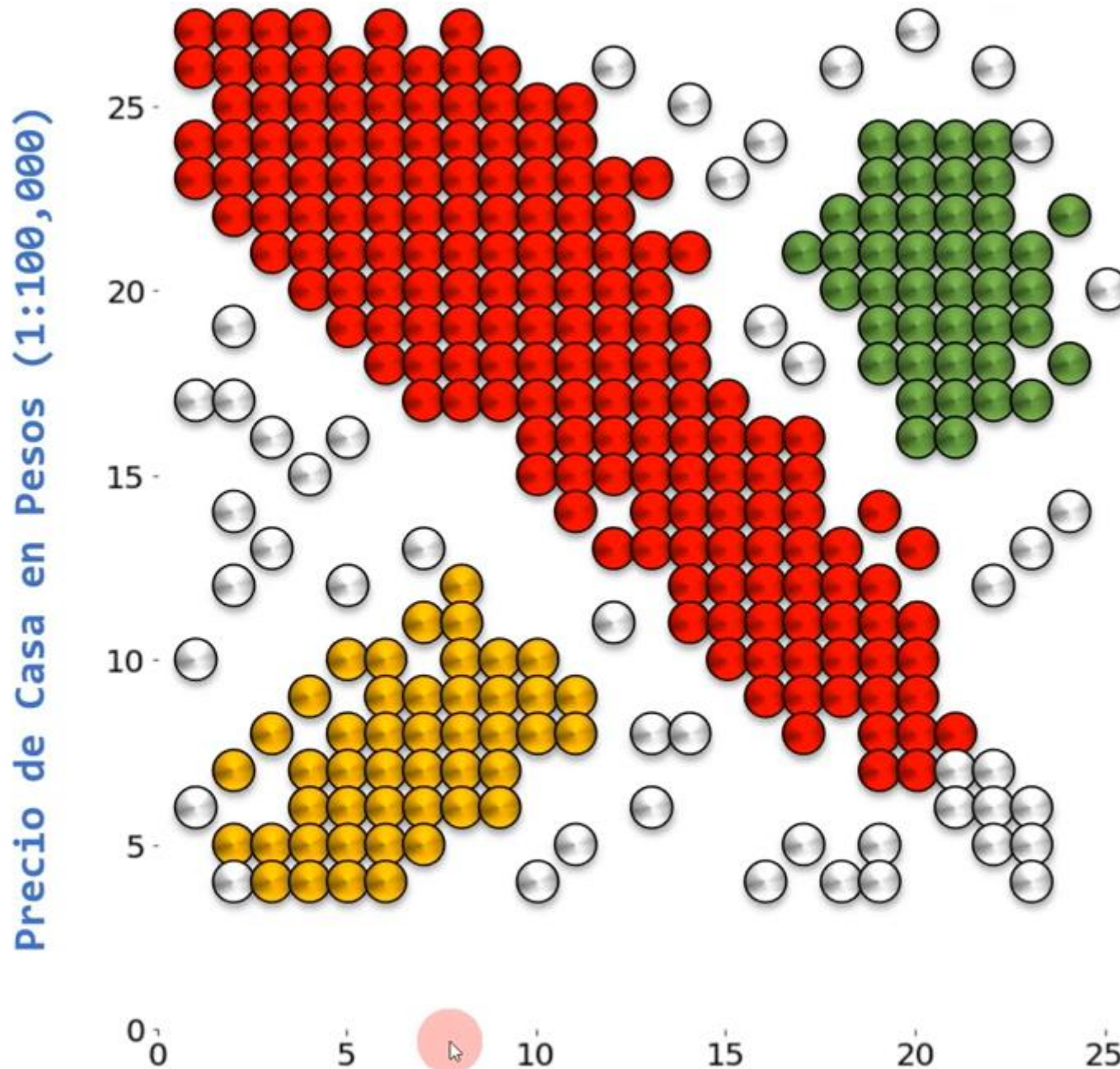
**Ahora, se
identifican
puntos RUIDO
alcanzables
desde CORES
y se unen al
Cluster**

Clustering – DBSCAN



Resultado de
añadir **puntos**
contiguos a los
CORES

Clustering – DBSCAN



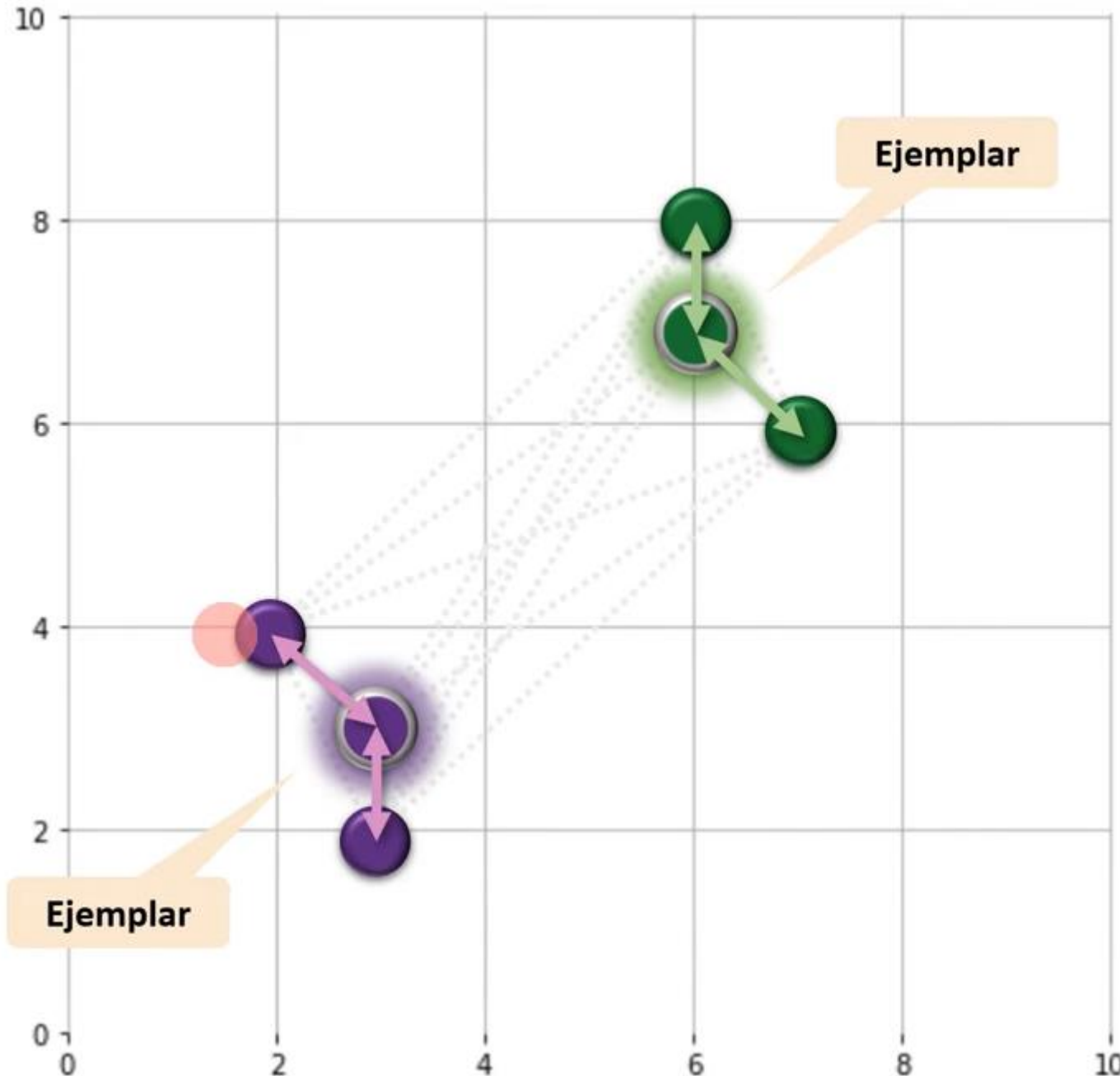
3 Clusters

Clustering – Propagación de Afinidad

Clustering – Propagación de afinidad

- El algoritmo se basa en la construcción de **matrices de similitud, responsabilidad y disponibilidad**.
- La **similitud** mide qué tan cercanos están los puntos entre sí,
- La **responsabilidad y la disponibilidad** reflejan la preferencia de un punto por otro como su "ejemplar" y cuán adecuado es un punto para ser un ejemplar, respectivamente.
- El "**factor de decaimiento**" en el contexto del algoritmo de Propagación de Afinidades es un parámetro crucial que ayuda a estabilizar las actualizaciones de las matrices de responsabilidad y disponibilidad a lo largo del proceso iterativo del algoritmo. Este factor puede tomar valores entre 0 y 1 y actúa como un coeficiente que controla la tasa a la que se actualizan estas matrices entre iteraciones, incorporando una porción del valor anterior de la matriz en cada nueva actualización.

Clustering – Propagación de afinidad



Affinity Propagation



Mensajes de Responsabilidad

Evidencia de qué tan adecuado es el **receptor** como **ejemplar**



Mensajes de Disponibilidad

Evidencia de qué tan adecuado es el **emisor** como **ejemplar**

Clustering – Propagación de afinidad

Affinity Propagation: Contexto y Datos

```
: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import AffinityPropagation

datos = {"alan" : [2, 4],
        "alberto" : [3, 3],
        "alex" : [3, 2],
        "zelda" : [7, 6],
        "zoila" : [6, 7],
        "zulema" : [6, 8]}

personas = set(datos.keys())
datos = pd.DataFrame(datos, index=["ahorro", "evaluacion"])
datos
```

```
: 
```

	alan	alberto	alex	zelda	zoila	zulema
ahorro	2	3	3	7	6	6
evaluacion	4	3	2	6	7	8

Clustering – Propagación de afinidad

Visualización de Datos

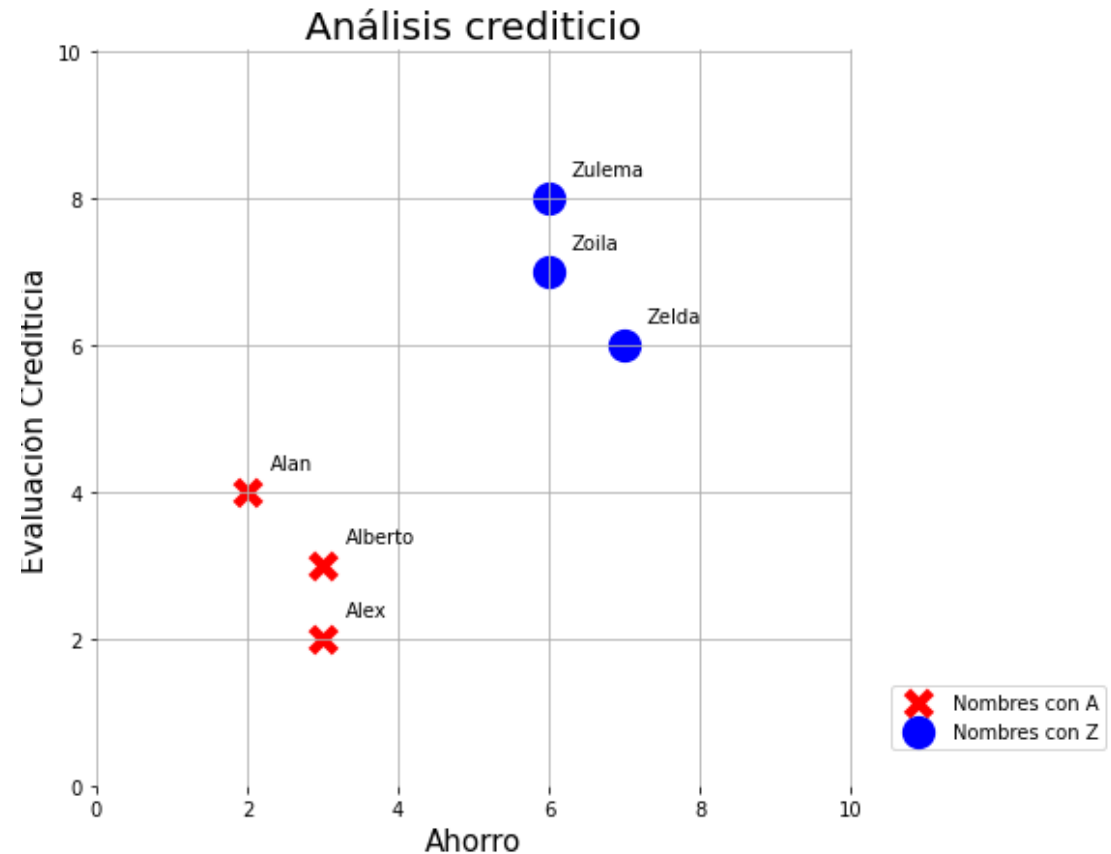
```
In [2]: plt.figure(figsize=(7, 7))
plt.title("Análisis crediticio", fontsize=20)

plt.scatter(datos.T[0:3]["ahorro"],
            datos.T[0:3]["evaluacion"],
            marker="x", s=150, color="red",
            linewidths=5, label="Nombres con A")

plt.scatter(datos.T[3:]["ahorro"],
            datos.T[3:]["evaluacion"],
            marker="o", s=150, color="blue",
            linewidths=5, label="Nombres con Z")

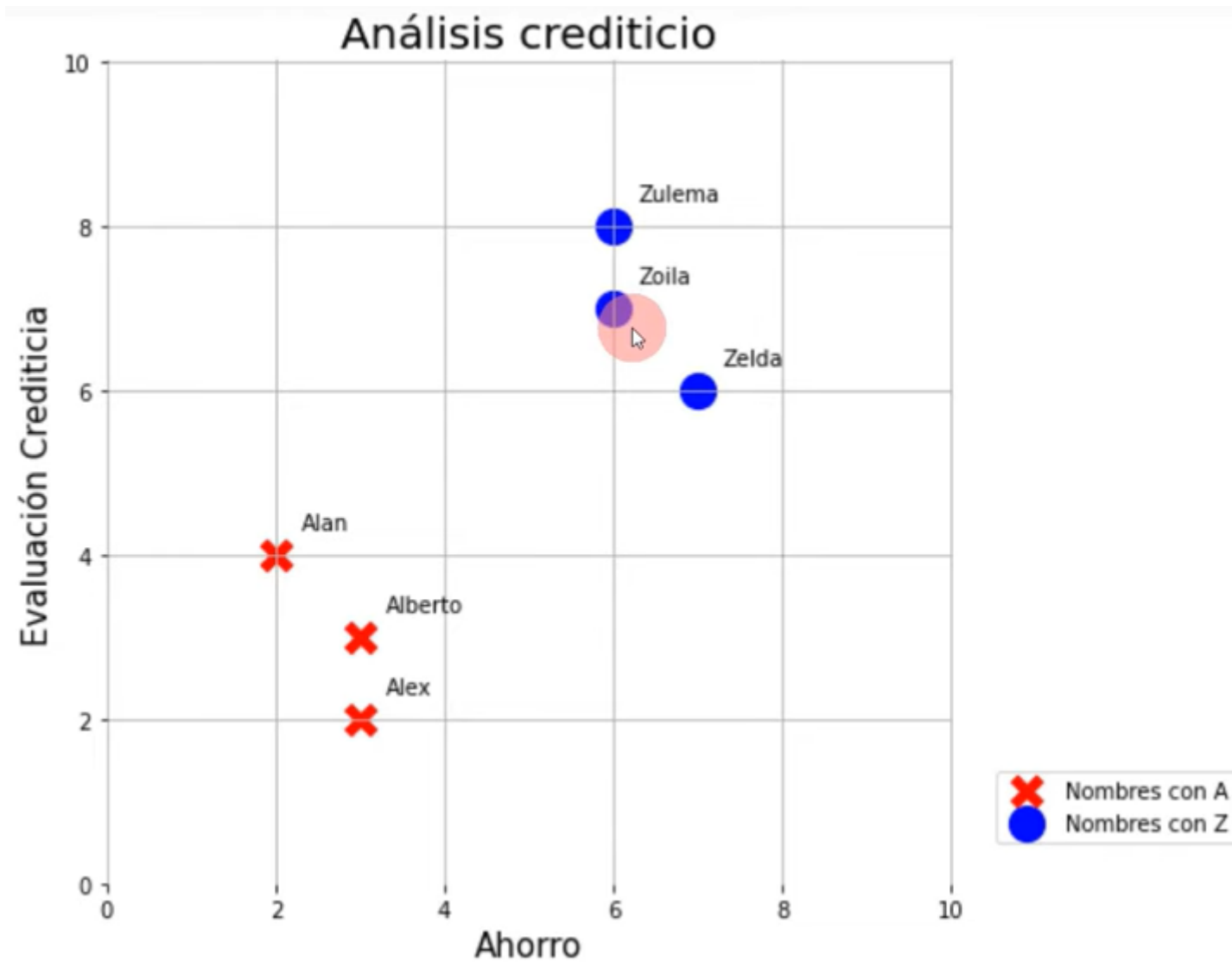
for i in range(len(datos.columns)):
    plt.text(datos[datos.columns[i]][0]+0.3,
            datos[datos.columns[i]][1]+0.3,
            datos.columns[i].capitalize())

plt.xlabel("Ahorro", fontsize=15)
plt.ylabel("Evaluación Crediticia", fontsize=15)
plt.legend(bbox_to_anchor=(1.35, 0.15))
plt.box(False)
plt.xlim((0, 10.01))
plt.ylim((0, 10.01))
plt.grid()
plt.show()
```



Affinity Propagation: Maximizar la similitud total entre los puntos y sus Ejemplares

Clustering – Propagación de afinidad



Clustering – Propagación de afinidad

Matrices para la Propagación de Afinidad

- **s : Similitud** (Entrada)
- **r : Responsabilidad**
- **d : Disponibilidad**
- **a : Asignaciones** (Salida)

Clustering – Propagación de afinidad

Similitud entre dos puntos i, k

Métrica: Distancia euclidiana al cuadrado

In [3]: datos

Out[3]:

	alan	alberto	alex	zelda	zoila	zulema
ahorro	2	3	3	7	6	6
evaluacion	4	3	2	6	7	8

```
In [5]: print("Similitud entre Alan y Alberto", -((datos["alan"] - datos["alberto"])**2).sum())  
print("Similitud entre Alan y Zulema", -((datos["alan"] - datos["zulema"])**2).sum())
```

```
Similitud entre Alan y Alberto -2  
Similitud entre Alan y Zulema -32
```

Clustering – Propagación de afinidad

Creación de Matriz de Similitud para $i \neq k$

```
In [6]: s = pd.DataFrame(0, columns=datos.columns, index=datos.columns)

for i in personas:
    for k in personas:
        s.loc[i, k] = -((datos[i] - datos[k])**2).sum()

s
```

Out[6]:

	alan	alberto	alex	zelda	zoila	zulema
alan	0	-2	-5	-29	-25	-32
alberto	-2	0	-1	-25	-25	-34
alex	-5	-1	0	-32	-34	-45
zelda	-29	-25	-32	0	-2	-5
zoila	-25	-25	-34	-2	0	-1
zulema	-32	-34	-45	-5	-1	0

Clustering – Propagación de afinidad

Preferencia a priori de cada punto como Ejemplar

```
In [7]: np.fill_diagonal(s.values, np.min(s.values))
```

s

Out[7]:

	alan	alberto	alex	zelda	zoila	zulema
alan	-45	-2	-5	-29	-25	-32
alberto	-2	-45	-1	-25	-25	-34
alex	-5	-1	-45	-32	-34	-45
zelda	-29	-25	-32	-45	-2	-5
zoila	-25	-25	-34	-2	-45	-1
zulema	-32	-34	-45	-5	-1	-45

Clustering – Propagación de afinidad

Mensajes de Disponibilidad

Evidencia de qué tan adecuado es el emisor como Ejemplar

```
In [8]: # Matriz de Disponibilidad
d = pd.DataFrame(0, columns=datos.columns, index=datos.columns)
d
```

Out[8]:

	alan	alberto	alex	zelda	zoila	zulema
alan	0	0	0	0	0	0
alberto	0	0	0	0	0	0
alex	0	0	0	0	0	0
zelda	0	0	0	0	0	0
zoila	0	0	0	0	0	0
zulema	0	0	0	0	0	0

Clustering – Propagación de afinidad

Mensajes de Responsabilidad

Evidencia de qué tan adecuado es el receptor como Ejemplar

```
In [ ]: # La responsabilidad se actualiza a partir de la similitud y la disponibilidad  
s
```

```
In [ ]: # Responsabilidad ["alan", "alberto"]  
  
print("Qué tan adecuado es Alberto como EJEMPLAR para Alan:", )  
  
# Responsabilidad ["alan", "zulema"]  
  
print("Qué tan adecuada es Zulema como EJEMPLAR para Alan:", )
```

Clustering – Propagación de afinidad

Mensajes de Responsabilidad

Evidencia de qué tan adecuado es el receptor como Ejemplar

```
In [9]: # La responsabilidad se actualiza a partir de la similitud y la disponibilidad
```

Out[9]:

	alan	alberto	alex	zelda	zoila	zulema
alan	-45	-2	-5	-29	-25	-32
alberto	-2	-45	-1	-25	-25	-34
alex	-5	-1	-45	-32	-34	-45
zelda	-29	-25	-32	-45	-2	-5
zoila	-25	-25	-34	-2	-45	-1
zulema	-32	-34	-45	-5	-1	-45

Clustering – Propagación de afinidad

Mensajes de Responsabilidad

Evidencia de qué tan adecuado es el receptor como Ejemplar

```
In [9]: # La responsabilidad se actualiza a partir de la similitud y la disponibilidads
```

Out[9]:

	alan	alberto	alex	zelda	zoila	zulema
alan	-45	-2	-5	-29	-25	-32
alberto	-2	-45	-1	-25	-25	-34
alex	-5	-1	-45	-32	-34	-45
zelda	-29	-25	-32	-45	-2	-5
zoila	-25	-25	-34	-2	-45	-1
zulema	-32	-34	-45	-5	-1	-45

Clustering – Propagación de afinidad

Mensajes de Responsabilidad

Evidencia de que tan adecuado es el receptor como Ejemplar

```
In [9]: # La responsabilidad se actualiza a partir de la similitud y la disponibilidad  
s
```

Out[9]:

	alan	alberto	alex	zelda	zoila	zulema
alan	-45	-2	-5	-29	-25	-32
alberto	-2	-45	-1	-25	-25	-34
alex	-5	-1	-45	-32	-34	-45
zelda	-29	-25	-32	-45	-2	-5
zoila	-25	-25	-34	-2	-45	-1
zulema	-32	-34	-45	-5	-1	-45

```
In [10]: # Responsabilidad ["alan", "alberto"]  
  
sim = s.loc["alan", "alberto"]  
  
sim_otros = s.loc["alan", ["alan", "alex", "zelda", "zoila", "zulema"]]  
  
dis_otros = d.loc["alan", ["alan", "alex", "zelda", "zoila", "zulema"]]
```

```
print("Qué tan adecuado es Alberto como EJEMPLAR para Alan:", sim - max(sim_otros + dis_otros))
```

```
sim = s.loc["alan", "zulema"]
```

```
sim_otros = s.loc["alan", ["alan", "alberto", "alex", "zelda", "zoila"]]
```

```
dis_otros = d.loc["alan", ["alan", "alberto", "alex", "zelda", "zoila"]]
```

```
# Responsabilidad ["alan", "zulema"]
```

```
print("Qué tan adecuada es Zulema como EJEMPLAR para Alan:", sim - max(sim_otros + dis_otros))
```

```
Qué tan adecuado es Alberto como EJEMPLAR para Alan: 3
```

```
Qué tan adecuada es Zulema como EJEMPLAR para Alan: 1-30
```

Clustering – Propagación de afinidad

Algoritmo Principal - Actualización de Matrices:

- **r** : Responsabilidad
- **d** : Disponibilidad
- **a** : Asignaciones

```
for i in range(10):
```

```
    factor = 0.5
```

```
# Actualización de Responsabilidades
```

```
r_anterior = r.copy()
```

```
for i in personas:
```

```
    for k in personas:
```

```
        elegibles = list(personas.difference({k}))
```

```
        r.loc[i, k] = s.loc[i, k] - max(s.loc[i, elegibles] + d.loc[i, elegibles])
```

```
# Actualización de Disponibilidades
```

```
d_anterior = d.copy()
```

```
for i in personas:
```

```
    for k in personas:
```

```
        if i == k:
```

```
            elegibles = list(personas.difference({i}))
```

```
            d.loc[k, k] = r.loc[elegibles, k][r.loc[elegibles, k] > 0].sum()
```

```
        else:
```

```
            elegibles = list(personas.difference({i, k}))
```

```
            d.loc[i, k] = min(0, r.loc[k, k] + r.loc[elegibles, k][r.loc[elegibles, k] > 0].sum())
```

```
# Actualización de Asignaciones
```

```
a = r + d
```

Clustering – Propagación de afinidad

Matriz de Asignaciones

```
In [11]: np.set_printoptions(precision=2)
print("Máximos", a.max(1).values)
print("Posición de los máximos", np.argmax(a.values, axis=1))
a.round(decimals=2)
```

Máximos [21.63 25.59 26.4 21.63 25.59 26.4]
Posición de los máximos [1 1 1 1 4 4 4]

Out[11]:

	alan	alberto	alex	zelda	zoila	zulema
alan	-31.67	21.63	-30.44	-54.04	-20.28	-55.87
alberto	-32.36	25.59	-31.23	-48.59	-18.82	-56.41
alex	-31.62	26.40	-30.64	-56.80	-29.03	-68.62
zelda	-54.04	-20.28	-55.87	-31.67	21.63	-30.44
zoila	-48.59	-18.82	-56.41	-32.36	25.59	-31.23
zulema	-56.80	-29.03	-68.62	-31.62	26.40	-30.64

Clustering – Propagación de afinidad

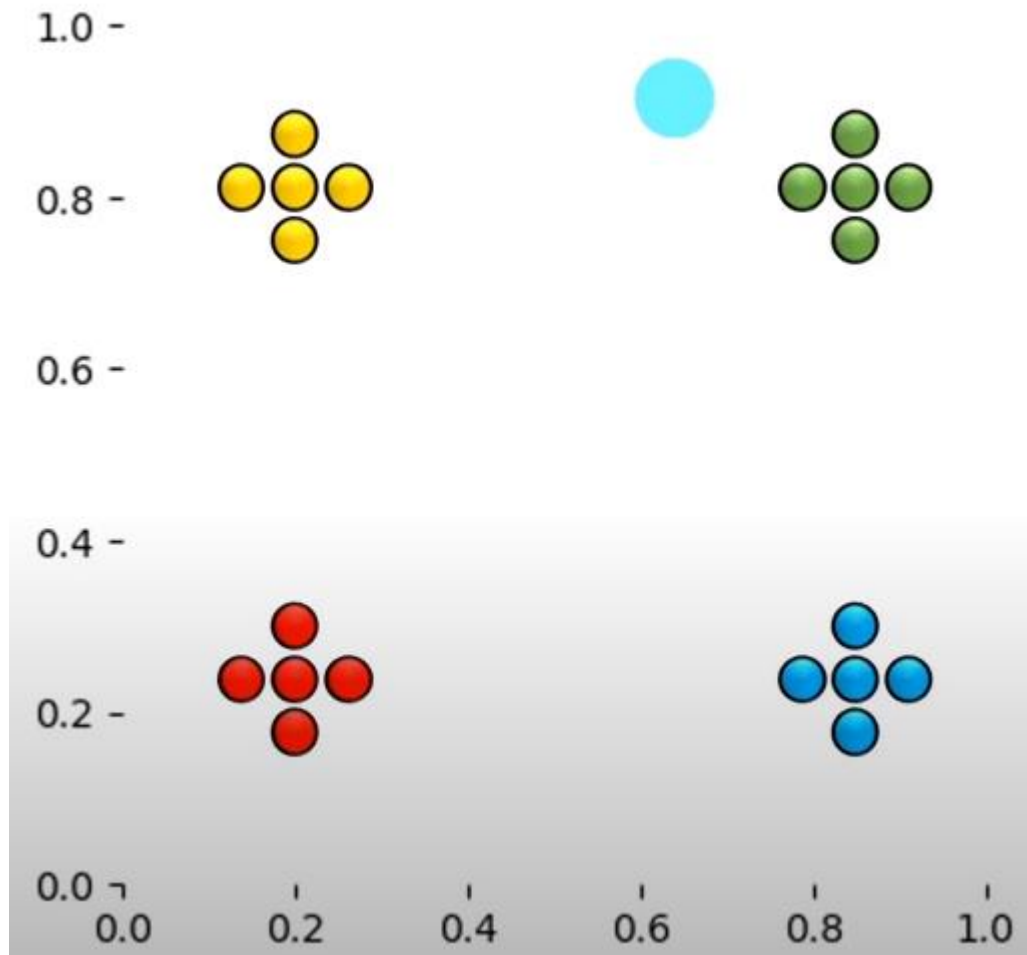
Propagación de afinidad con Scikit-Learn

```
In [12]: clustering = AffinityPropagation(random_state=None).fit(datos.T.values)
print("Etiquetas de Clusters:", clustering.labels_)
print("Índices de Centroides:", clustering.cluster_centers_indices_)
```

```
Etiquetas de Clusters: [0 0 0 1 1 1]
Índices de Centroides: [1 4]
```

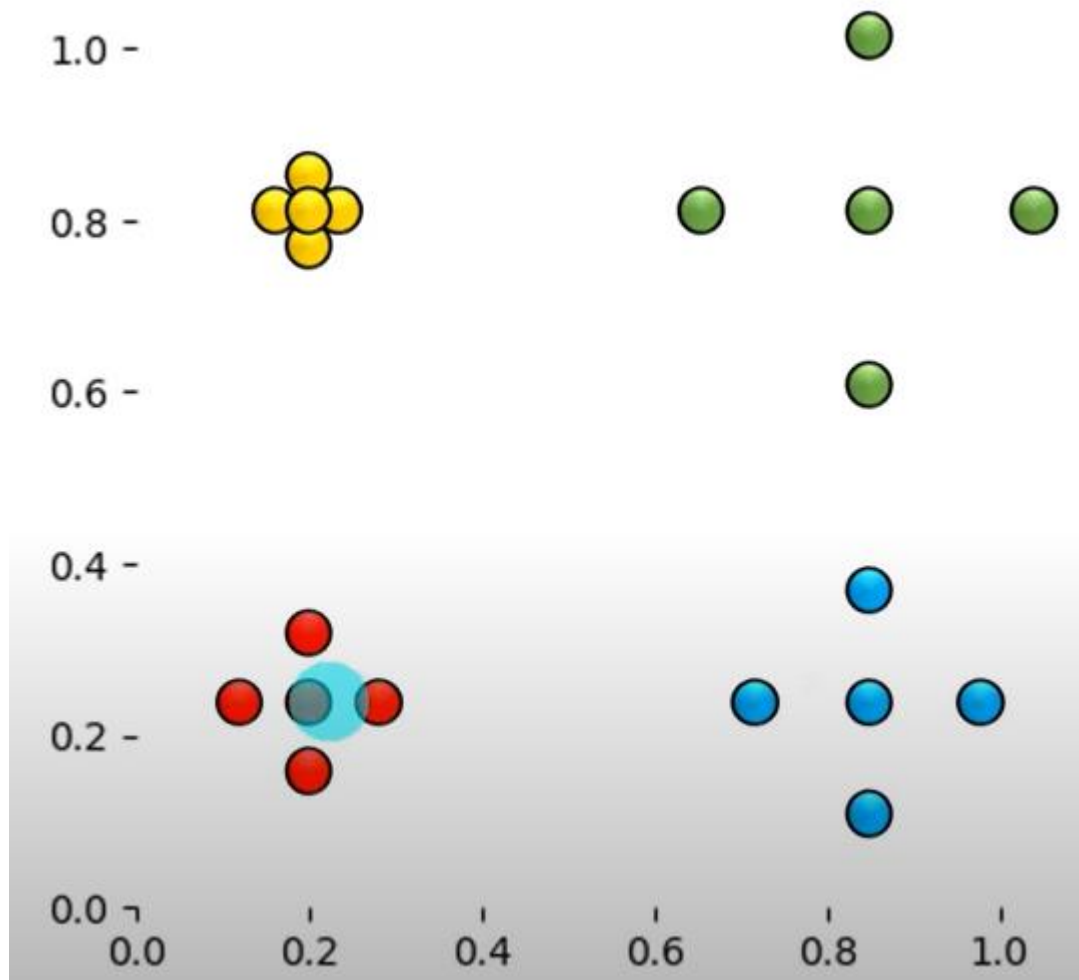
Clustering – Métricas

Clustering – Métricas



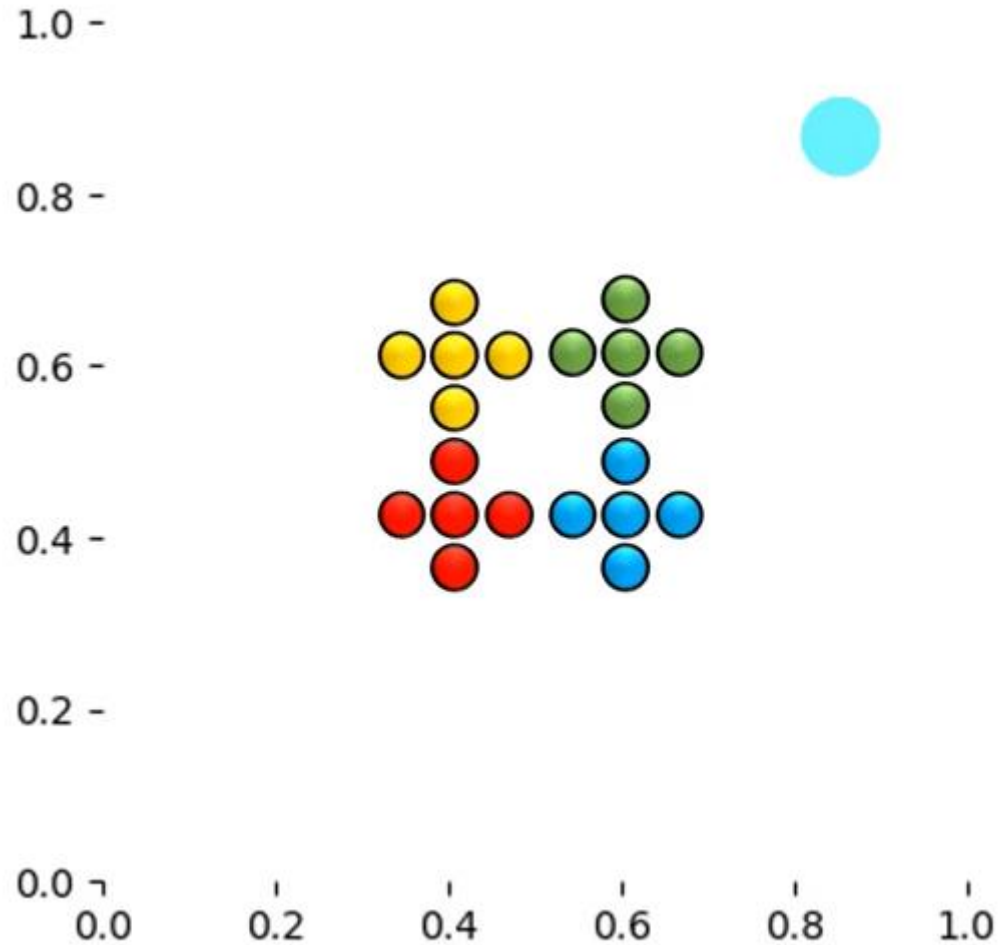
Cohesión
Intra-cluster

Clustering – Métricas



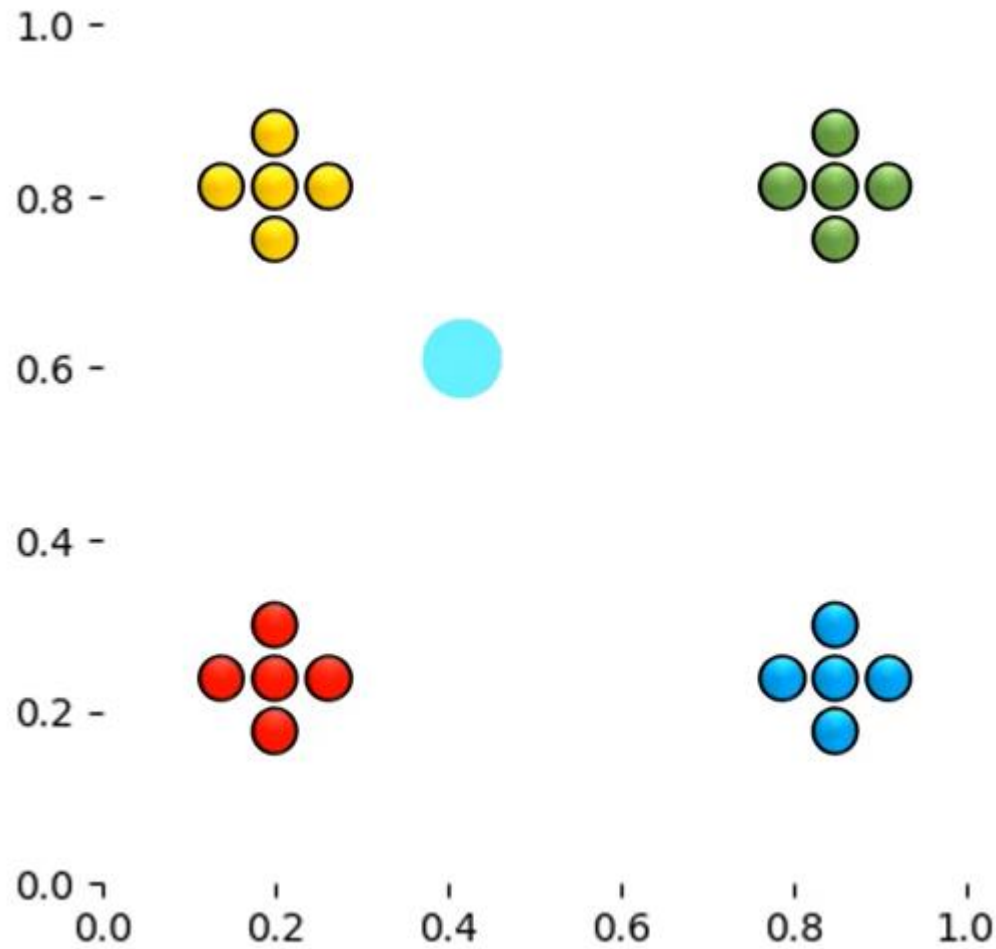
Cohesión
Intra-cluster

Clustering – Métricas



Separación
Inter-cluster

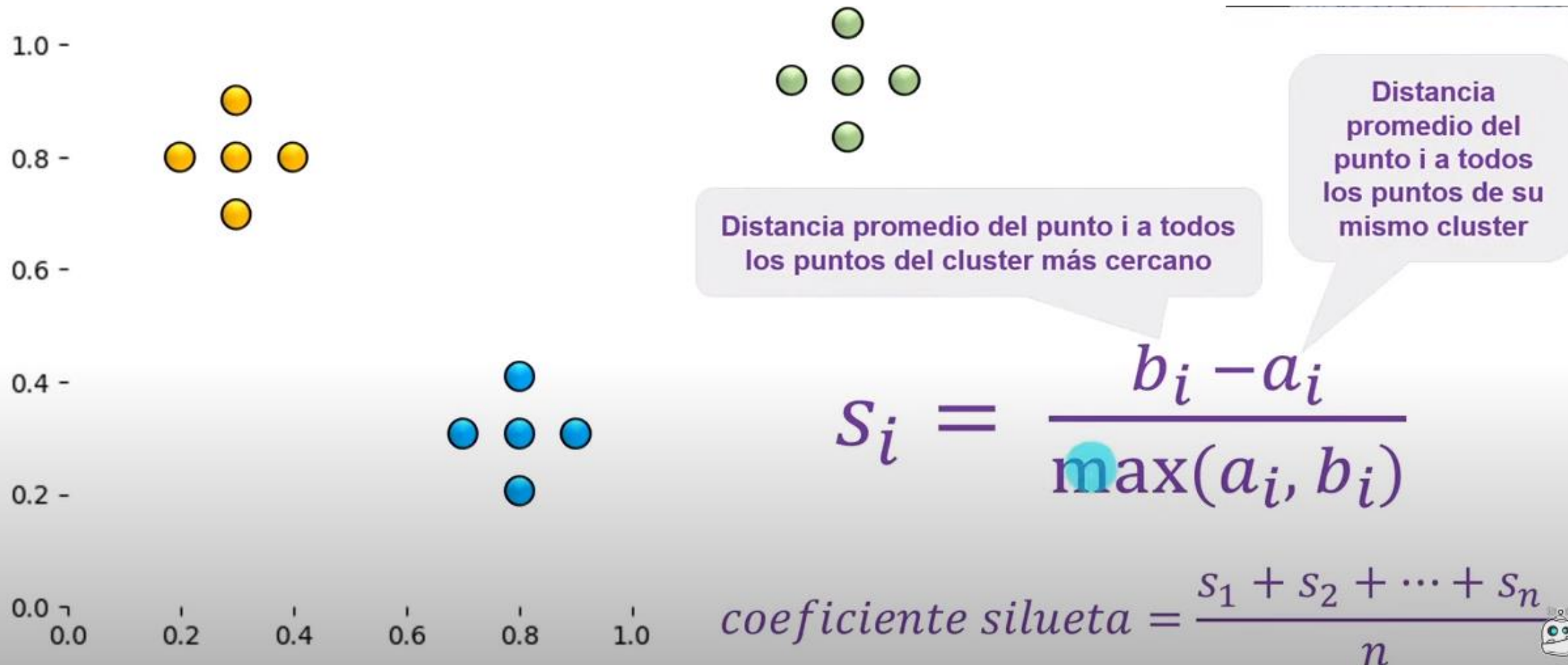
Clustering – Métricas



Separación
Inter-cluster

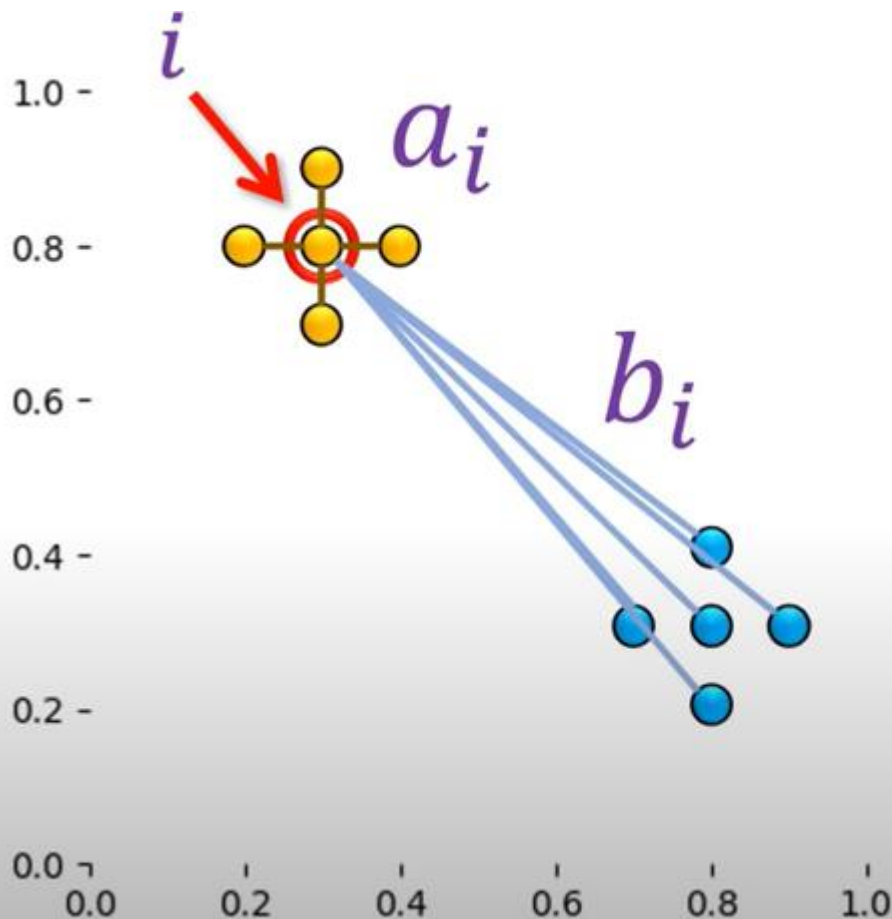
Clustering – Métricas

Coeficiente Silueta



Clustering – Métricas

Coeficiente Silueta



Distancia promedio del punto i a todos los puntos del cluster más cercano

Distancia promedio del punto i a todos los puntos de su mismo cluster

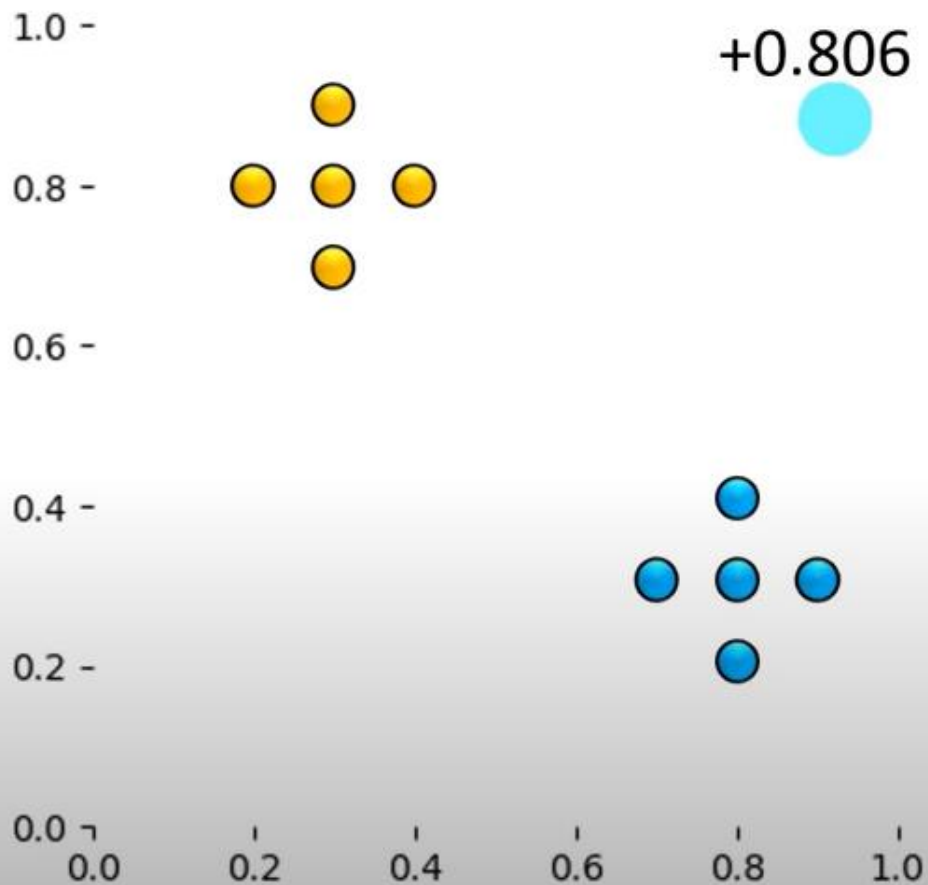
$$S_i = \frac{b_i - a_i}{\max(a_i, b_i)}$$

$$\text{coeficiente silueta} = \frac{s_1 + s_2 + \dots + s_n}{n}$$



Clustering – Métricas

Coeficiente Silueta



$$S_i = \frac{b_i - a_i}{\max(a_i, b_i)}$$

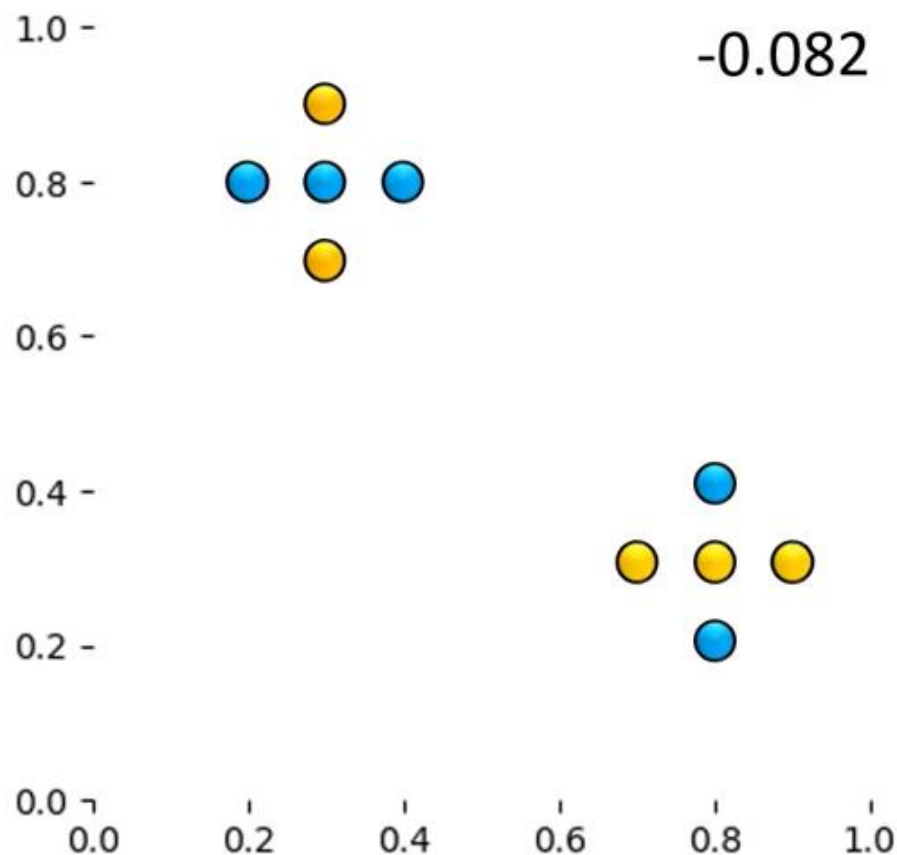
Si coeficiente silueta se aproxima a **1**

$$b > a$$

Clusters bien formados

Clustering – Métricas

Coeficiente Silueta



$$S_i = \frac{b_i - a_i}{\max(a_i, b_i)}$$

Si coeficiente silueta se aproxima a **0**

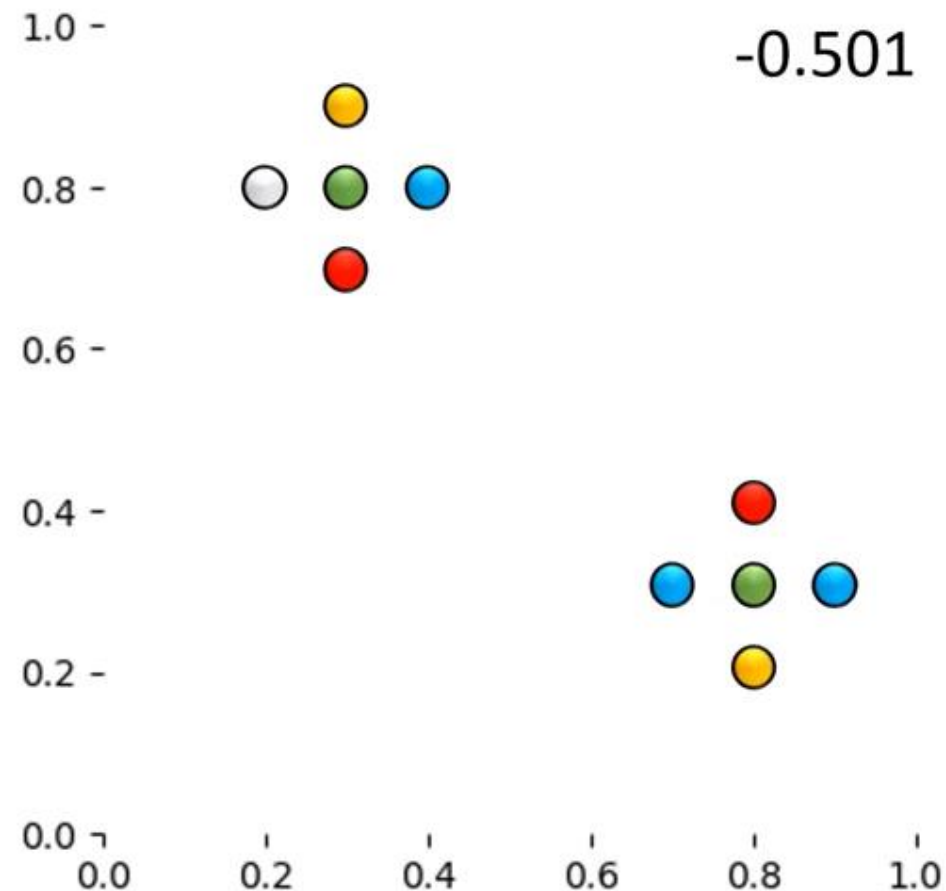
$$b \approx a$$

Clusters superpuestos



Clustering – Métricas

Coeficiente Silueta



$$S_i = \frac{b_i - a_i}{\max(a_i, b_i)}$$

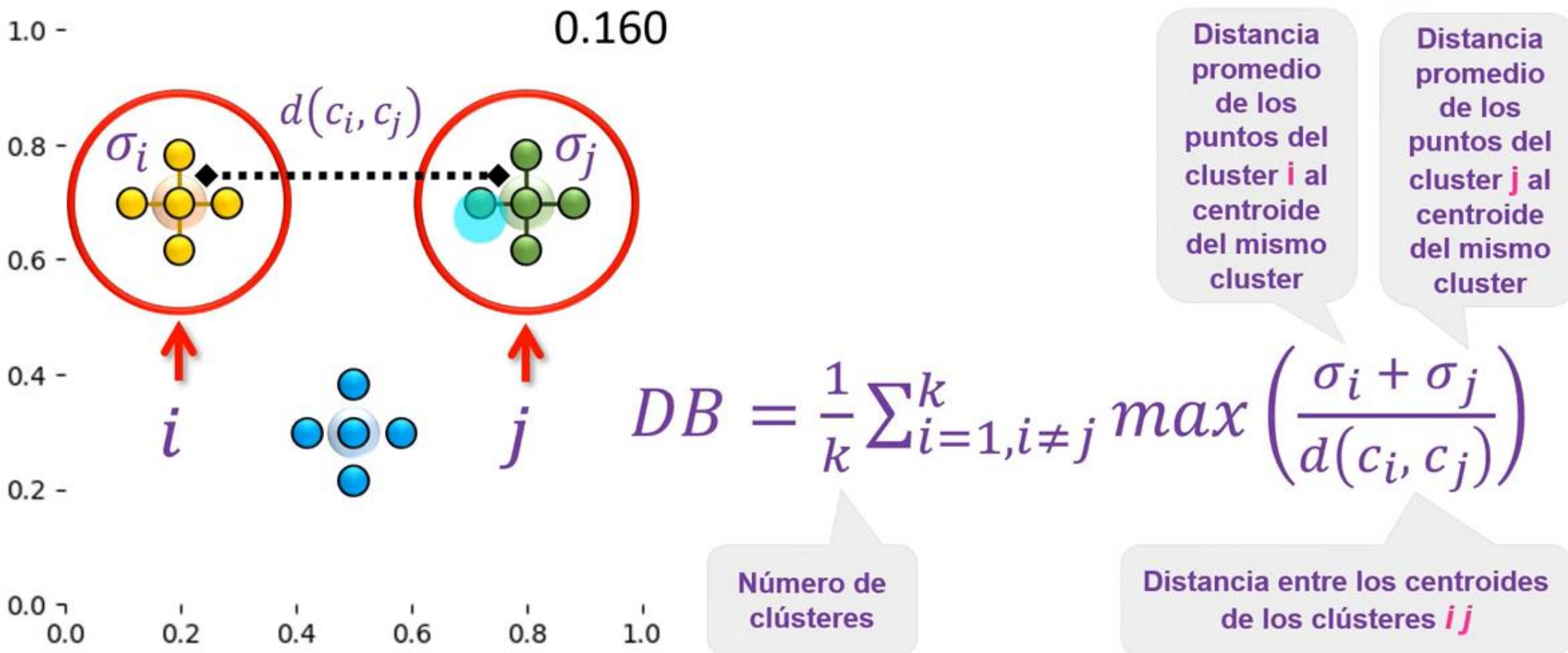
Si coeficiente silueta se aproxima a -1


$$b < a$$

Asignación errónea

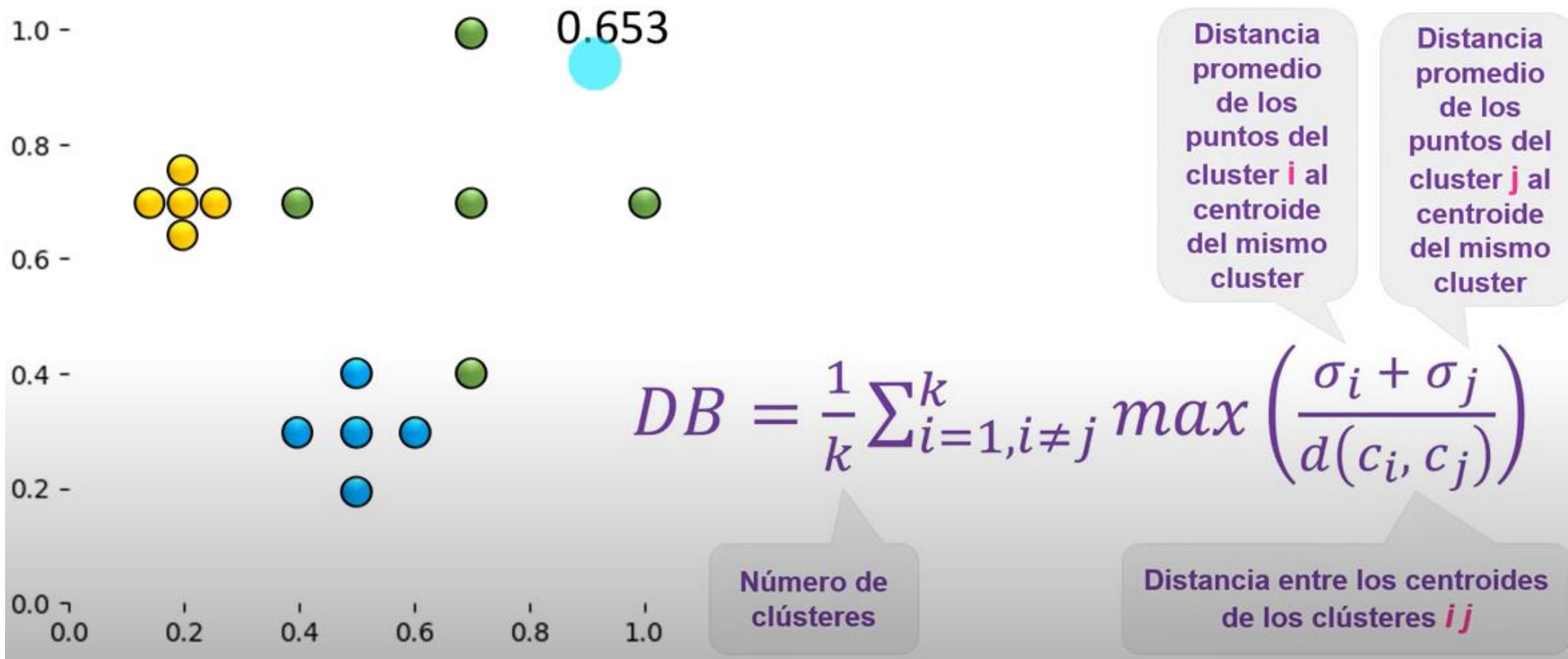
Clustering – Métricas

Índice Davies-Bouldin



Clustering – Métricas

Índice Davies-Bouldin



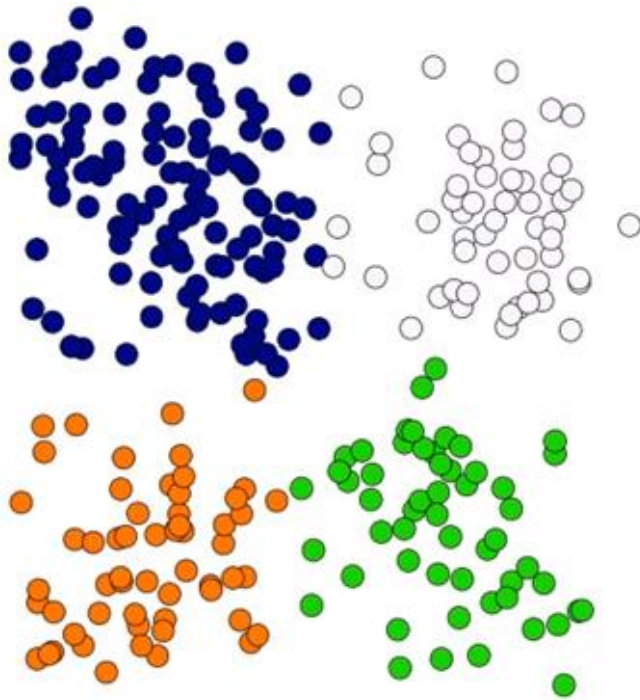
Clustering – Métricas

Comparación

Silueta = 0.53

Davies-Bouldin = 0.59

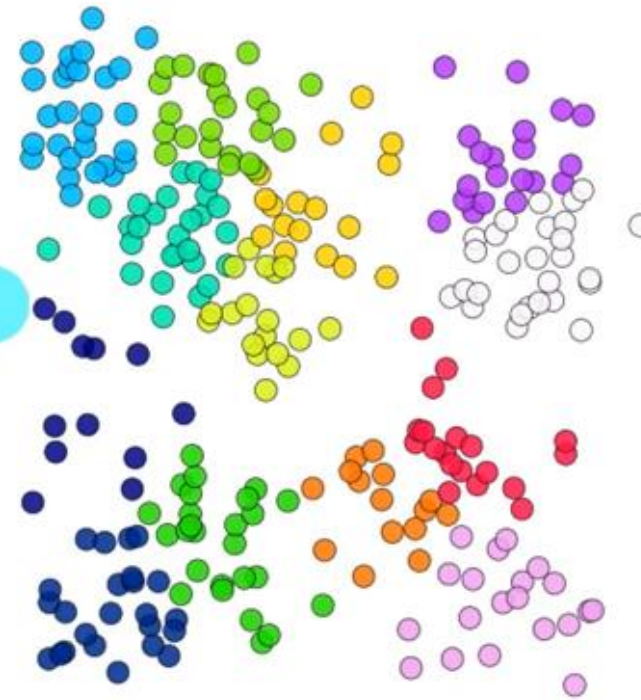
K-Medias



Silueta = 0.36

Davies-Bouldin = 0.86

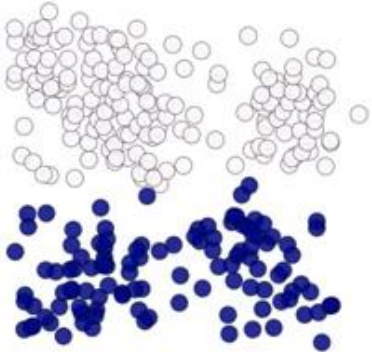
Propagación de Afinidad



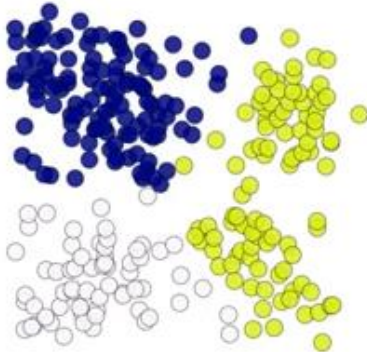
Clustering – Métricas

Comparación

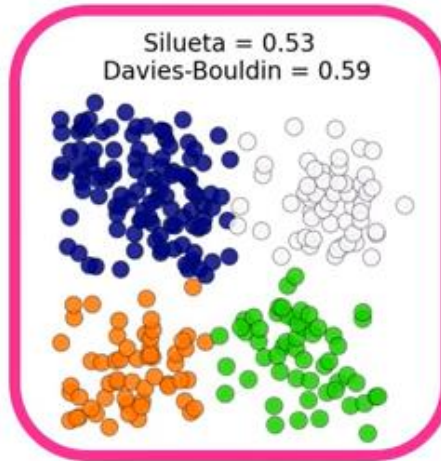
Silueta = 0.41
Davies-Bouldin = 1.04



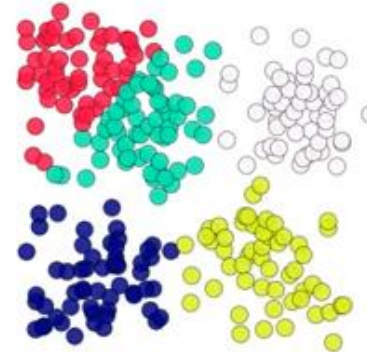
Silueta = 0.48
Davies-Bouldin = 0.74



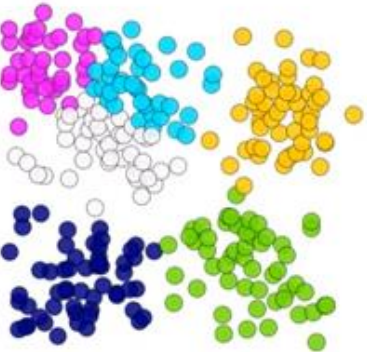
Silueta = 0.53
Davies-Bouldin = 0.59



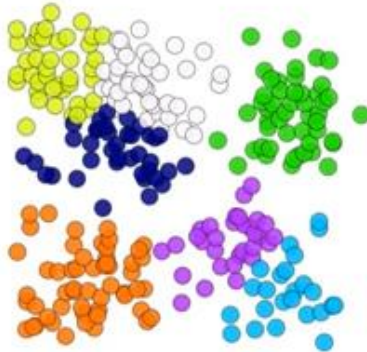
Silueta = 0.48
Davies-Bouldin = 0.75



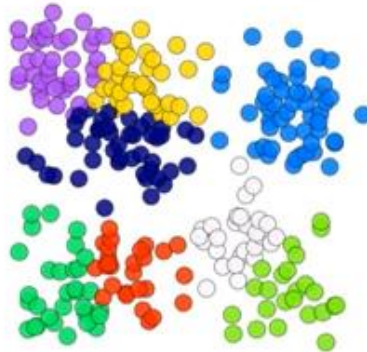
Silueta = 0.46
Davies-Bouldin = 0.76



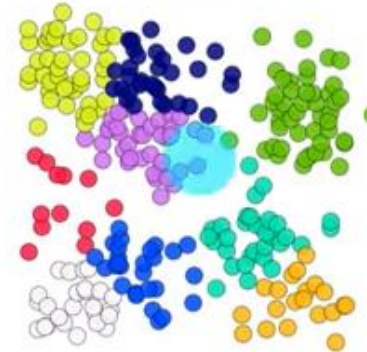
Silueta = 0.43
Davies-Bouldin = 0.83



Silueta = 0.39
Davies-Bouldin = 0.89



Silueta = 0.42
Davies-Bouldin = 0.82



K-Medias
con
diferentes
valores de
K

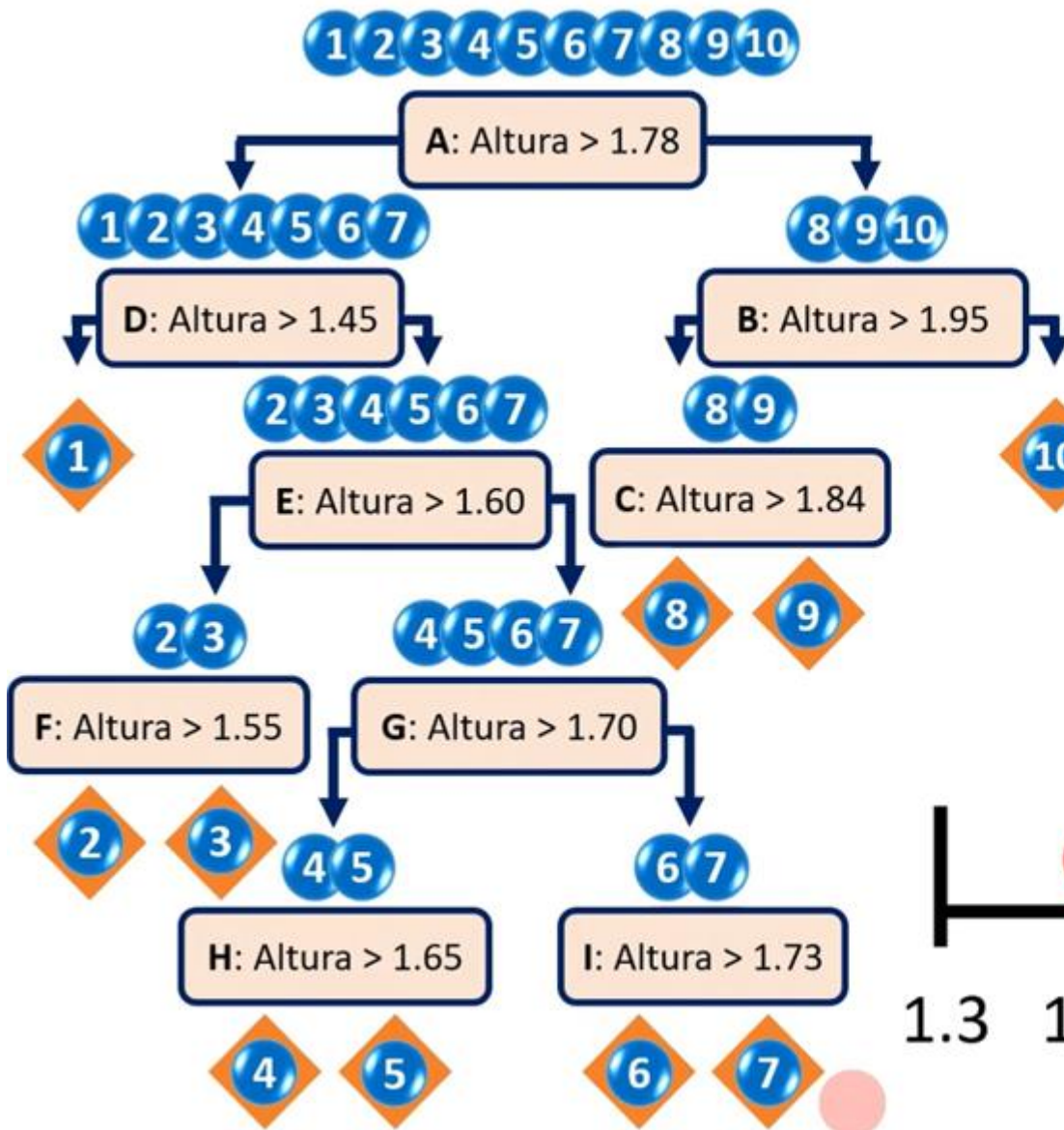
Detección de anomalías

Bosques de aislamiento

DetECCIÓN DE ANOMALÍAS – BOSQUES DE AISLAMIENTO

- Estos bosques funcionan bajo el principio de que los datos anómalos son más fáciles de aislar que los datos normales.
- Al realizar cortes aleatorios en las características de los datos, los puntos anómalos tienden a ser separados en menos pasos que los datos regulares.
- Cada árbol en el bosque aísla un dato al dividir el espacio de características hasta que cada punto queda en su propio nodo terminal.
- La anomalía de un punto se mide en función de la profundidad promedio en la que se aísla en los árboles del bosque: **los puntos anómalos se aíslan más cerca de la raíz del árbol**, mientras que los normales se encuentran más profundos.
- Para mejorar la precisión, se crean **múltiples árboles y se promedia** el resultado.
- Además, se utiliza **un parámetro de contaminación (% de datos anómalos que hay)** para ajustar el umbral de decisión entre datos normales y anómalos, lo cual permite manejar la expectativa de la proporción de anomalías en el conjunto de datos. Esta información de cuanta contaminación puede haber nos viene de un experto en el dominio.
- **El muestreo** en los bosques de aislamiento (Isolation Forest) juega un papel crucial para manejar eficientemente grandes volúmenes de datos. En cada iteración para construir un árbol de aislamiento dentro del bosque, se selecciona aleatoriamente un subconjunto de datos del conjunto de datos total

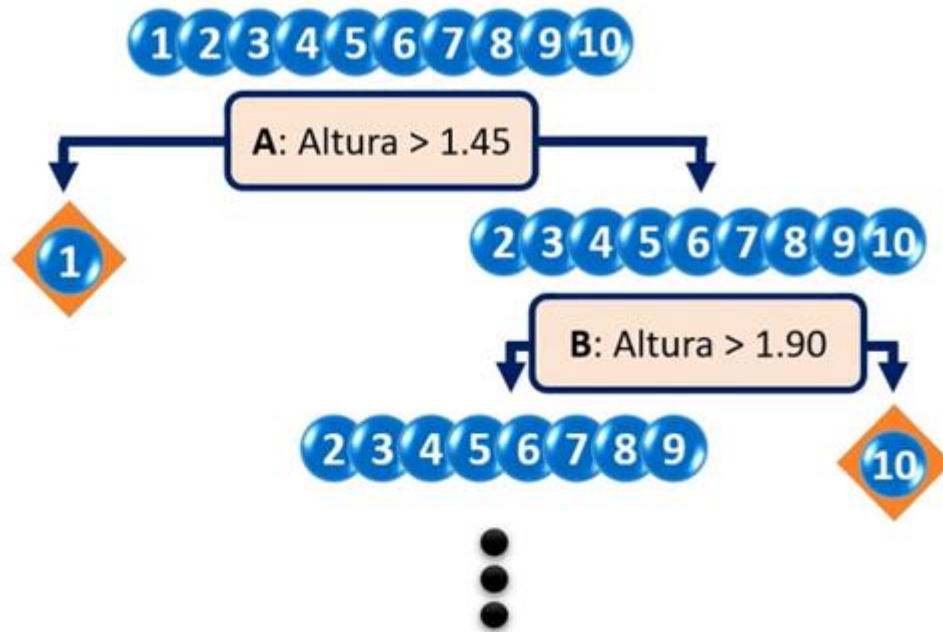
Detección de anomalías – Bosques de aislamiento



Creación de un iTree de un Bosque de Aislamiento



Detección de anomalías – Bosques de aislamiento



Creación de otro iTree de un Bosque de Aislamiento

¡Se crean muchos iTrees!

En promedio:

- Datos normales en ramas largas
- Datos **anómalos** en **ramas cortas**



Detección de anomalías – Bosques de aislamiento

- **Matriz de confusión**
 - Precisión
 - Recall (Recuperación o Sensibilidad)
 - F1-Score
- **Área bajo la curva ROC** (Receiver Operating Characteristic, AUC-ROC)